

MySQL тригер

Тригер в MySQL е набор от SQL изрази, които се намират в системен каталог. **Това е специален тип съхранена процедура, която се извиква автоматично в отговор на събитие** . Всеки тригер е свързан с таблица, която се активира на всеки DML израз като **INSERT, UPDATE** или **DELETE** .

Тригерът се нарича специална процедура, защото не може да бъде извикан директно като съхранена процедура. Основната разлика между тригера и процедурата е, че тригерът се извиква автоматично, когато се направи събитие за промяна на данни спрямо таблица. За разлика от това, съхранената процедура трябва да бъде извикана изрично.

Обикновено **тригерите са два типа** според стандарта SQL : тригери на ниво ред и тригери на ниво израз.

Row-Level Trigger /Тригер на ниво ред/: Това е тригер, който се активира за всеки ред чрез оператор за задействане, като например вмъкване, актуализиране или изтриване. Например, ако таблица е вмъкнала, актуализирала или изтрила няколко реда, тригерът за ред се задейства автоматично за всеки ред, засегнат от израза за вмъкване, актуализиране или изтриване .

Statement-Level Trigger /Тригер на ниво оператор/: Това е тригер, който се задейства веднъж за всяко събитие, което се случва в таблица, независимо колко реда са вмъкнати, актуализирани или изтрити.

***ЗАБЕЛЕЖКА:** Трябва да знаем, че MySQL не поддържа тригери на ниво оператор. Той осигурява поддръжка само за тригери на ниво ред.*

Защо имаме нужда/използваме тригери в MySQL?

Имаме нужда/използваме тригери в MySQL поради следните характеристики:

- Тригерите ни помагат да налагаме бизнес правила.
- Тригерите ни помагат да валидираме данните дори преди те да бъдат вмъкнати или актуализирани.
- Тригерите ни помагат да поддържаме регистър на записите, като поддържане на одитни пътеки в таблици.
- SQL тригерите предоставят алтернативен начин за проверка на целостта на данните.
- Тригерите предоставят алтернативен начин за изпълнение на планираната задача.
- Triggers увеличава производителността на SQL заявките, тъй като не е необходимо да се компилира всеки път, когато заявката се изпълнява.
- Тригерите намаляват кода от страна на клиента, което спестява време и усилия.
- Тригерите ни помагат да мащабираме нашето приложение в различни платформи.
- Тригерите са лесни за поддръжка.

Ограничения при използването на тригери в MySQL

- MySQL тригерите не позволяват използването на всички проверки; те предоставят само разширени валидации. **Например** , можем да използваме ограниченията NOT NULL, UNIQUE, CHECK и FOREIGN KEY за прости проверки.

- Тригерите се извикват и изпълняват невидимо от клиентското приложение. Следователно не е лесно да се отстрани какво се случва в слоя база данни.
- Тригерите могат да увеличат натоварването на сървъра на базата данни.

Можем да дефинираме максимум шест вида действия или събития под формата на тригери:

1. **Before Insert / Преди вмъкване** : Активира се преди вмъкването на данни в таблицата.
2. **After Insert / След вмъкване** : Активира се след вмъкване на данни в таблицата.
3. **Before Update / Преди актуализация** : Активира се преди актуализацията на данните в таблицата.
4. **After Update / След актуализиране** : Активира се след актуализиране на данните в таблицата.
5. **Before Delete / Преди изтриване** : Активира се преди данните да бъдат премахнати от таблицата.
6. **After Delete / След изтриване** : Активира се след изтриване на данни от таблицата.

Когато използваме оператор, който не използва заявка INSERT, UPDATE или DELETE за промяна на данните в таблица, тригерите, свързани с тригера, няма да бъдат извикани.

Конвенции за именуване

Конвенциите за именуване са набор от правила, които следваме, за да дадем подходящи уникални имена. Спестява ни време, за да поддържаме работата организирана и разбираема. Следователно **трябва да използваме уникално име за всеки тригер, свързан с таблица** . Въпреки това е добра практика да имате едно и също име на тригер, дефинирано за различни таблици.

Следната конвенция за именуване трябва да се използва за именуване на тригера в MySQL:

```
(BEFORE | AFTER) table_name (INSERT | UPDATE | DELETE)
```

по този начин

Trigger Activation Time: BEFORE | AFTER / Време за активиране на тригера: ПРЕДИ | СЛЕД/

Trigger Event: INSERT | UPDATE | DELETE /Задействащо събитие: INSERT | АКТУАЛИЗАЦИЯ | ИЗТРИВАНЕ/

Как да създадете тригери в MySQL?

Можем да използваме оператора **CREATE TRIGGER** за създаване на нов тригер в MySQL. По-долу е синтаксисът за създаване на тригер в MySQL:

```
CREATE TRIGGER trigger_name
(AFTER | BEFORE) (INSERT | UPDATE | DELETE)
ON table_name FOR EACH ROW
BEGIN
--variable declarations
--trigger code
END;
```

Тригер за създаване на MySQL

Можем да създадем нов тригер в MySQL, като използваме командата `CREATE TRIGGER`. Целта е да се гарантира, че имаме привилегии за действие, докато използваме командата `CREATE TRIGGER`. Следва основният синтаксис за създаване на тригер:

```
CREATE TRIGGER trigger_name trigger_time trigger_event  
  
ON table_name FOR EACH ROW  
  
BEGIN  
  
    --variable declarations  
  
    --trigger code  
  
END;
```

Параметъри

Синтаксисът на тригера за създаване съдържа следните параметри:

trigger_name: Това е името на тригера, който искаме да създадем. Трябва да бъде написан след оператора **CREATE TRIGGER**. Целта е да се гарантира, че името на тригера трябва да бъде уникално в рамките на схемата.

trigger_time: Това е времето за действие на тригера, което трябва да бъде **BEFORE** или **AFTER**. Това е задължителният параметър при дефиниране на тригер. Той показва, че тригерът ще бъде извикан преди или след всяка модификация на ред в таблицата.

trigger_event: Това е типът име на операцията, която активира тригера. Това може да бъде операция **INSERT**, **UPDATE** или **DELETE**. Тригерът може да извика само едно събитие наведнъж. Ако искаме да дефинираме тригер, който се извиква от множество събития, трябва да дефинираме множество тригери и по един за всяко събитие.

table_name: Това е името на таблицата, към която е свързан тригерът. Трябва да се напише след ключовата дума **ON**. Ако не посочим името на таблицата, тригерът няма да съществува.

BEGIN END Блок: Накрая ще посочим оператора за изпълнение, когато тригерът е активиран. Ако искаме да изпълним множество изрази, ще използваме блока **BEGIN END**, който съдържа набор от заявки за дефиниране на логиката за тригера.

Тялото на тригера има достъп до стойностите на колоната, които са засегнати от DML оператора (DML - Език за манипулиране на данни): *DML командите се занимават с операции върху данни, налични в базата данни, а DML командите съставляват по-голямата част от SQL изразите..* Модификаторите **NEW** и **OLD** се използват за разграничаване на стойностите на колоните **BEFORE** и **AFTER** изпълнението на DML оператора. Можем да използваме името на колоната с модификатори **NEW** и **OLD** като **OLD.col_name** и **NEW.col_name**. **OLD.column_name** указва колоната на съществуващ ред преди актуализирането или изтриването. **NEW.col_name** указва колоната на нов ред, който ще бъде вмъкнат, или съществуващ ред, след като бъде актуализиран.

Да предположим например , че искаме да актуализираме името на колоната **message_info** с помощта на тригера. В тялото на тригера можем да получим достъп до стойността на колоната преди актуализацията като **OLD.message_info** и новата стойност **NEW.message_info**.

Можем да разберем наличието на СТАРИ и НОВИ модификатори с таблицата по-долу:

Trigger Event	OLD	NEW
INSERT	No	Yes
UPDATE	Yes	Yes
DELETE	Yes	н No е

Пример за задействане на MySQL

Нека започнем да създаваме тригер в MySQL , който прави модификации в таблицата на служителите. Първо, ще създадем нова таблица, наречена **employee**, като изпълним следния оператор:

```
CREATE TABLE employee(  
    name varchar(45) NOT NULL,  
    occupation varchar(35) NOT NULL,  
    working_date date,  
    working_hours varchar(10)  
);
```

След това изпълнете оператора по-долу, за да **попълните записите** в таблицата на служителите:

```
INSERT INTO employee VALUES  
( 'Robin', 'Scientist', '2020-10-04', 12),  
( 'Warner', 'Engineer', '2020-10-04', 10),  
( 'Peter', 'Actor', '2020-10-04', 13),  
( 'Marco', 'Doctor', '2020-10-04', 14),  
( 'Brayden', 'Teacher', '2020-10-04', 12),  
( 'Antonio', 'Business', '2020-10-04', 11);
```

След това изпълнете **оператора SELECT** , за да проверите вмъкнатия запис:

```
SELECT * FROM employee;
```

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM employee;
+-----+-----+-----+-----+
| name   | occupation | working_date | working_hours |
+-----+-----+-----+-----+
| Robin  | Scientist  | 2020-10-04   | 12            |
| Warner | Engineer   | 2020-10-04   | 10            |
| Peter  | Actor      | 2020-10-04   | 13            |
| Marco  | Doctor     | 2020-10-04   | 14            |
| Brayden | Teacher    | 2020-10-04   | 12            |
| Antonio | Business   | 2020-10-04   | 11            |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

След това ще създадем **тригер BEFORE INSERT**. Този тригер се извиква автоматично, вмъкнете **work_hours = 0**, ако някой се опита да вмъкне **work_hours < 0**.

```
DELIMITER //

Create Trigger before_insert_empworkinghours

BEFORE INSERT ON employee FOR EACH ROW

BEGIN

IF NEW.working_hours < 0 THEN SET NEW.working_hours = 0;

END IF;

END //
```

Ако тригерът е създаден успешно, ще получим резултата, както следва:

```
MySQL 8.0 Command Line Client
mysql> DELIMITER //
mysql> Create Trigger before_insert_empworkinghours
-> BEFORE INSERT ON employee FOR EACH ROW
-> BEGIN
-> IF NEW.working_hours < 0 THEN SET NEW.working_hours = 0;
-> END IF;
-> END //
Query OK, 0 rows affected (0.28 sec)
```

Сега можем да използваме следните изрази, за да извикаме този тригер:

```
INSERT INTO employee VALUES ('Markus', 'Former', '2020-10-08', 14);

INSERT INTO employee VALUES ('Alexander', 'Actor', '2020-10-012', -13);
```

След изпълнение на горния оператор, ще получим изхода, както следва:

```
MySQL 8.0 Command Line Client

mysql> INSERT INTO employee VALUES
-> ('Markus', 'Former', '2020-10-08', 14);
Query OK, 1 row affected (0.18 sec)

mysql> INSERT INTO employee VALUES
-> ('Alexander', 'Actor', '2020-10-012', -13);
Query OK, 1 row affected (0.16 sec)

mysql> SELECT * FROM employee;
+-----+-----+-----+-----+
| name      | occupation | working_date | working_hours |
+-----+-----+-----+-----+
| Robin     | Scientist  | 2020-10-04   | 12            |
| Warner    | Engineer   | 2020-10-04   | 10            |
| Peter     | Actor      | 2020-10-04   | 13            |
| Marco     | Doctor     | 2020-10-04   | 14            |
| Brayden   | Teacher    | 2020-10-04   | 12            |
| Antonio   | Business   | 2020-10-04   | 11            |
| Markus    | Former     | 2020-10-08   | 14            |
| Alexander | Actor      | 2020-10-12   | 0             |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

В този изход можем да видим, че при вмъкване на отрицателните стойности в колоната `work_hours` на таблицата автоматично ще се запълни нулевата стойност чрез тригер.

MySQL Show/List Triggers

Тригерът за показване или списък е много необходим, когато имаме много бази данни, които съдържат различни таблици. Понякога имаме едни и същи имена на тригери в много бази данни; тази заявка играе важна роля в този случай. Можем да получим информацията за задействане в сървъра на базата данни, като използваме израза по-долу. Този оператор връща всички тригери във всички бази данни:

```
SHOW TRIGGERS;
```

Необходими са следните стъпки, за да получите списъка с всички тригери:

Стъпка 1: Отворете командния ред на MySQL и влезте в сървъра на базата данни, като използвате паролата, която сте създали по време на инсталирането на MySQL . След успешна връзка можем да изпълним всички SQL изрази.

Стъпка 2: След това изберете конкретната база данни, като използвате командата по-долу:

```
USE database_name;
```

Стъпка 3: Накрая изпълнете командата **SHOW TRIGGERS**.

Нека го разберем с примера, даден по-долу. Да предположим, че имаме база данни с име **"mysqltestdb"** , която съдържа много таблици. След това изпълнете оператора по-долу, за да изброите тригерите :

```
USE mysqltestdb;
```

```
SHOW TRIGGERS;
```

Резултат:

```
MySQL 8.0 Command Line Client
mysql> USE mysqltestdb;
Database changed
mysql> SHOW TRIGGERS;
+-----+-----+-----+-----+-----+-----+
| Trigger | Timing | Event | Table | Statement | Definer |
| character_set_client | collation_connection | Database Collation |
+-----+-----+-----+-----+-----+-----+
| before_insert_empworkinghours | INSERT | employee | BEGIN
IF NEW.working_hours < 0 THEN SET NEW.working_hours = 0;
END IF;
END | BEFORE | 2020-11-13 14:49:05.83 | STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION | root@localhost | cp850
| cp850_general_ci | utf8mb4_0900_ai_ci |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Ако искаме да покажем или изброим информацията за задействане в конкретна база данни от текущата база данни без превключване, MySQL ни позволява да използваме клаузата **FROM** или **IN** , последвана от името на базата данни.

```
SHOW TABLES IN database_name;
```

Тази операция може да се напише и като:

```
SHOW TABLES FROM database_name;
```

Когато изпълним горните оператори, ще получим същия резултат.

Показване на тригери с помощта на съпоставяне на шаблони

Show Triggers Using Pattern Matching

MySQL също предоставя опция за клауза **LIKE**, която ни позволява да филтрираме името на тригера, като използваме различно съвпадение на шаблони. Синтаксисът за използване на съвпадение на шаблони с командата show trigger:

```
SHOW TRIGGERS LIKE pattern;
```

OR,

```
SHOW TRIGGERS FROM database_name LIKE pattern;
```

Ако искаме да изброим/покажем имена на тригери въз основа на конкретно условие за търсене, можем да използваме клаузата **WHERE** , както следва:

```
SHOW TRIGGERS WHERE search_condition;
```

OR,

```
SHOW TRIGGERS FROM database_name WHERE search_condition;
```

Пример

Да предположим, че искаме да покажем всички тригери, които принадлежат на таблицата **на employee**, изпълнете оператора:

```
SHOW TRIGGERS FROM mysqltestdb WHERE table = 'employee';
```

Ще получим резултата, както следва:

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_	collation_conn	Database Collation
before_insert_empworkinghou	INSERT	employee	BEGIN IF NEW.working_hours < 0 THEN SET NEW.working_hours = 0; END IF; END	BEFORE	2020-11-13 14:49:05.83	STRICT_TRANS_TABLES,NC	root@localhost	cp850	cp850_general_	utf8mb4_0900

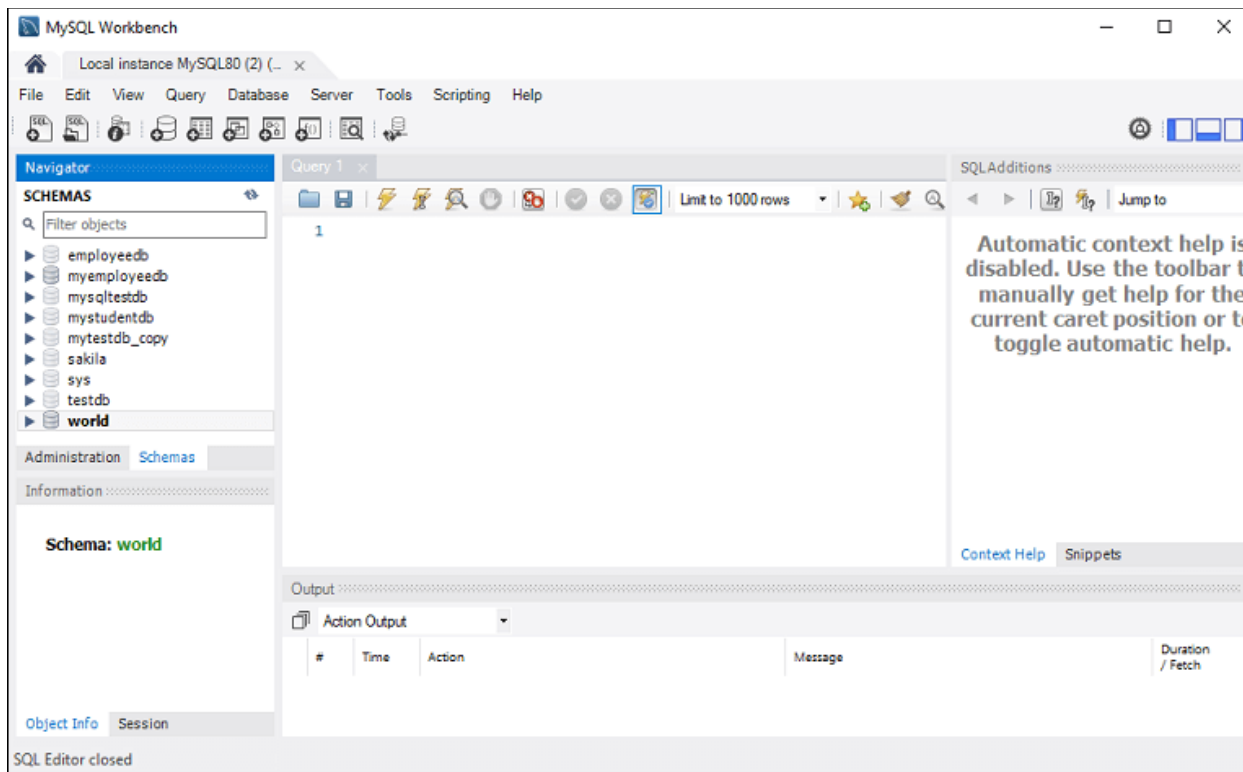
ЗАБЕЛЕЖКА: *Трябва да се отбележи, че трябва да имаме SUPER privilege привилегия, за да изпълним командата SHOW TRIGGERS.*

Операторът **show trigger** съдържа няколко колони в резултата. Нека обясним всяка колона подробно.

- **Trigger / Тригер:** Това е името на тригера, който искаме да създадем и трябва да бъде уникален в рамките на схемата.
- **Event / Събитие:** Това е типът име на операция, която извиква тригера. Това може да бъде операция INSERT, UPDATE или DELETE.
- **Table / Таблица:** Това е името на таблицата, към която принадлежи тригерът.
- **Statement / Изявление:** Това е тялото на тригера, което съдържа логика за тригера, когато се активира.
- **Timing / Време:** Това е времето за активиране на тригера, BEFORE или AFTER. Той показва, че тригерът ще бъде извикан преди или след всеки ред от модификации, който се появява в таблицата.
- **Created / Създаден:** Представява часа и датата, когато е създаден тригерът.
- **sql_mode:** Показва SQL_MODE, когато тригерът се изпълни.
- **Definer / Дефинитор:** Това е името на потребителски акаунт, който е създал тригера и трябва да бъде във формат 'user_name'@'host_name'.
- **character_set_client:** Това беше стойността на сесията на системната променлива character_set_client, когато тригерът беше създаден.
- **collation_connection:** Това беше стойността на сесията на системната променлива character_set_client, когато тригерът беше създаден.
- **Database Collation / Съпоставяне на база данни:** Определя правилата, които сравняват и подреждат символния низ. Това е сравняването на базата данни, към която принадлежи тригерът.

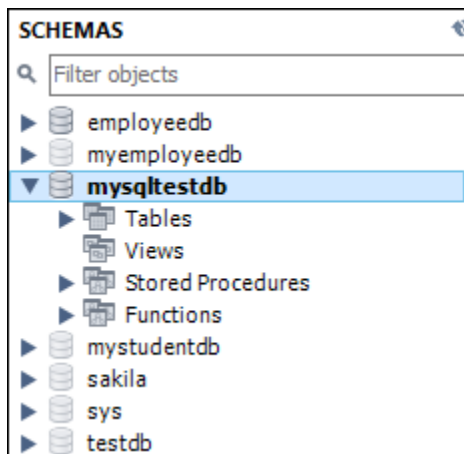
Как да покажа тригери в MySQL workbench?

За да покажем тригер с помощта на този инструмент, първо трябва да стартираме MySQL Workbench и да влезем с потребителското име и паролата, които сме създали по-рано. Ще получим екрана, както следва:

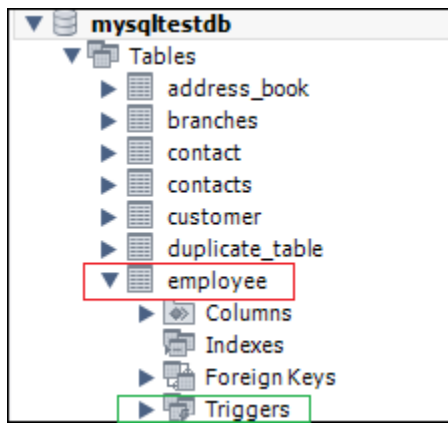


Сега изпълнете следните стъпки, за да покажете задействия:

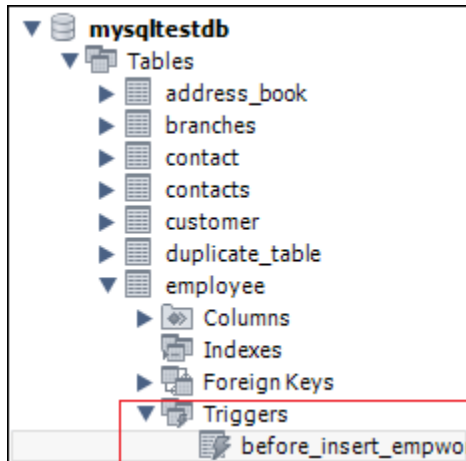
1. Отидете в раздела Navigation и щракнете върху **менюто Shemas** , което съдържа всички налични бази данни в MySQL сървъра.
2. Изберете базата данни (например **mysqltestdb**), щракнете два пъти върху нея и покажете **подменюто**, съдържащо таблици, изгледи, функции и съхранени процедури.



3. Щракнете върху **подменюто Tables** и изберете таблицата, за която сте създали тригер.



4. Щраквайки върху **подменюто Triggers** , можем да видим всички тригери, свързани с избраната таблица.



MySQL DROP Trigger

Можем да пуснем/изтрием/премахнем тригер в MySQL с помощта на оператора **DROP TRIGGER**. Трябва да сте много внимателни, докато премахвате тригера от таблицата. Тъй като след като сме изтрили тригера, той не може да бъде възстановен. Ако тригер не бъде намерен, операторът DROP TRIGGER извежда грешка.

MySQL ни позволява да пуснем/изтрием/премахнем тригер главно по два начина:

1. MySQL клиент на командния ред
2. MySQL Workbench

MySQL клиент на командния ред

Можем да премахнем съществуващ тригер от базата данни, като използваме оператора DROP TRIGGER със следния синтаксис:

```
DROP TRIGGER [IF EXISTS] [schema_name.]trigger_name;
```

Параметри

Параметрите, използвани в синтаксиса на тригера за изпускане, са обяснени, както следва:

Параметър	Описания
Trigger_name	Това е името на тригер, който искаме да премахнем от сървъра на базата данни. Това е задължителен параметър.
Schema_name	Това е името на базата данни, към която принадлежи тригерът. Ако пропуснем този параметър, операторът ще премахне тригера от текущата база данни.
IF_EXISTS	Това е незадължителен параметър, който условно премахва тригерите само ако съществуват на сървъра на базата данни.

Ако премахнем тригера, който не съществува, ще получим грешка. Въпреки това, ако сме посочили клаузата **IF EXISTS**, MySQL дава **ЗАБЕЛЕЖКА** вместо грешка.

Трябва да се отбележи, че трябва да имаме привилегии на **TRIGGER**, преди да изпълним оператора **DROP TRIGGER** за таблицата, свързана с тригера. Също така, премахването на таблица автоматично ще изтрие всички тригери, свързани с таблицата.

Пример за задействане на **MySQL DROP**

Нека да видим как можем да премахнем тригера, свързан с таблицата. Първо ще **покажем всички тригери**, налични в избраната база данни, като използваме израза по-долу:

```
SHOW TRIGGERS IN employeeedb;
```

След като изпълним оператора, можем да видим, че има два тригера, наречени **before_update_salaries** и **sales_info_before_update**.

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer
before_update_salaries	UPDATE	salary_account	BEGIN IF new.amount < old.a...	BEFORE	2020-11-2...	STRICT_TRA...	root@localhost
sales_info_BEFORE_UPDATE	UPDATE	sales_info	BEGIN DECLARE error_msg VARC...	BEFORE	2020-11-2...	STRICT_TRA...	root@localhost

Ако искаме да премахнем тригера **before_update_salaries**, изпълнете оператора:

```
DROP TRIGGER employeeedb.before_update_salaries;
```

Той успешно ще изтрие тригер от базата данни. Ако изпълним горния оператор отново, той ще върне съобщение за грешка. Вижте резултата:

```
MySQL 8.0 Command Line Client

mysql> DROP TRIGGER employeeedb.before_update_salaries;
Query OK, 0 rows affected (0.20 sec)

mysql> DROP TRIGGER employeeedb.before_update_salaries;
ERROR 1360 (HY000): Trigger does not exist
```

Ако изпълним горния оператор отново с клауза **IF EXISTS**, той ще върне предупредителното съобщение, вместо да генерира грешка. Вижте резултата:

DROP TRIGGER IF EXISTS employeeedb.before_update_salaries;

```
MySQL 8.0 Command Line Client

mysql> DROP TRIGGER IF EXISTS employeeedb.before_update_salaries;
Query OK, 0 rows affected, 1 warning (0.12 sec)
```

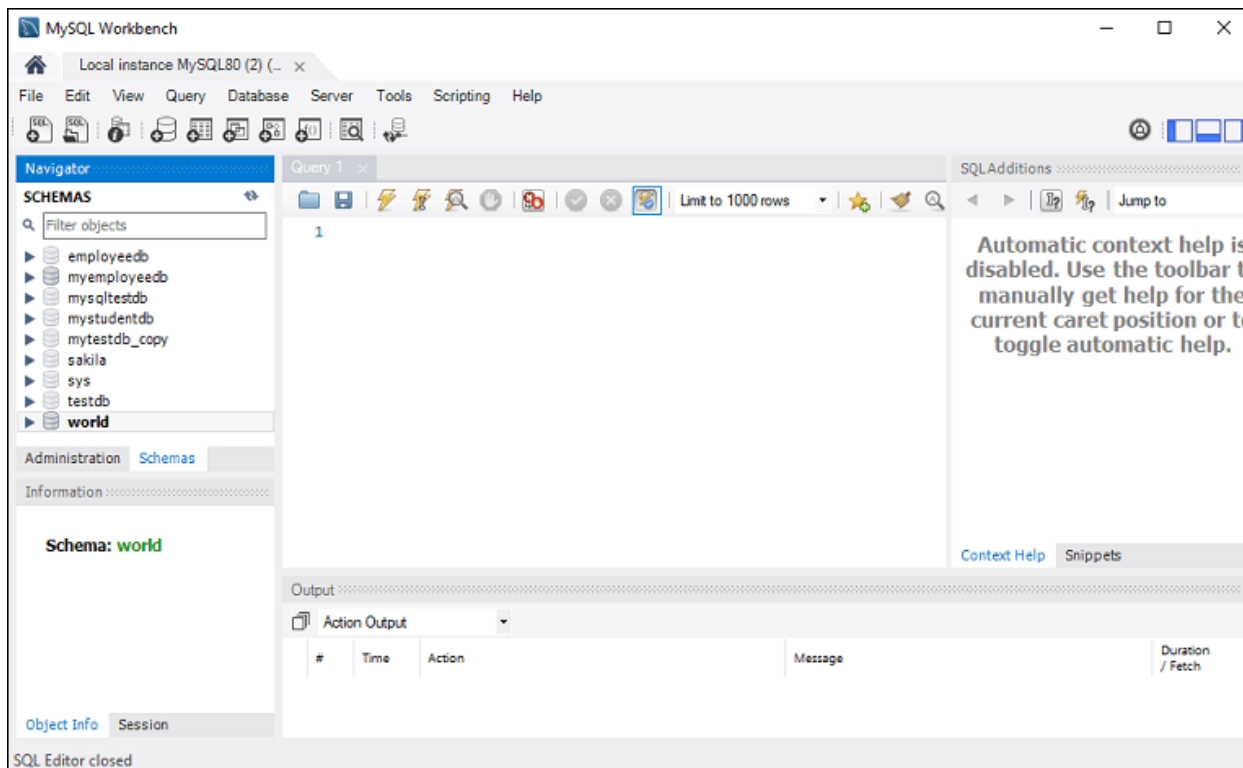
Можем да изпълним оператора **SHOW WARNING**, който генерира **ЗАБЕЛЕЖКА** за несъществуващ тригер, когато използваме IF EXISTS. Вижте резултата:

```
MySQL 8.0 Command Line Client

mysql> SHOW WARNINGS;
+-----+-----+-----+
| Level | Code | Message                               |
+-----+-----+-----+
| Note  | 1360 | Trigger does not exist               |
+-----+-----+-----+
1 row in set (0.00 sec)
```

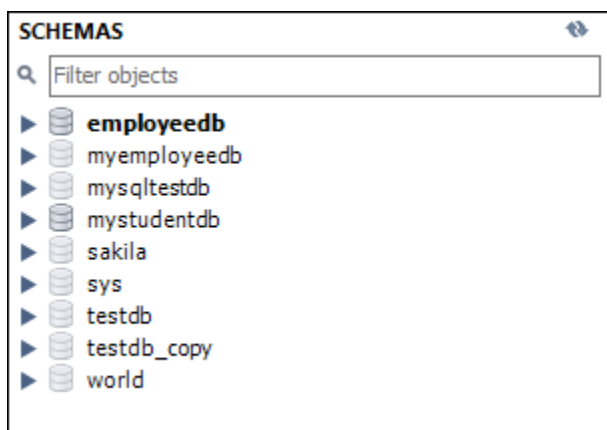
Как да пуснете тригер в MySQL workbench?

За да създадем тригер в работната среда, първо **стартираме MySQL Workbench** и влизаме с потребителското име и паролата. Ще получим потребителския интерфейс, както следва:

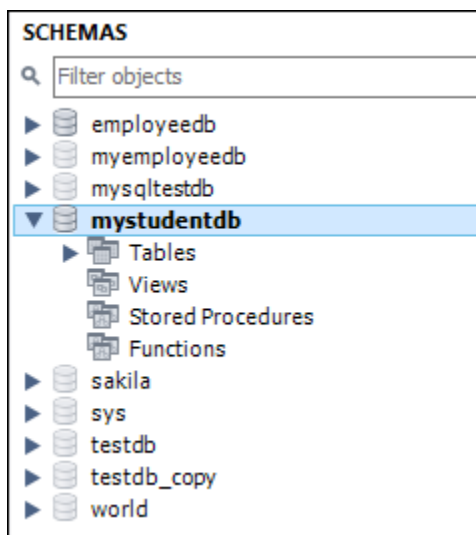


Сега направете следните стъпки, за да изтриете или унищожите тригер, свързан с таблицата:

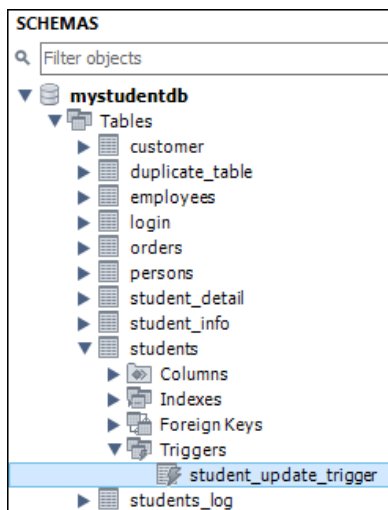
1. Отидете в раздела **Navigator** и щракнете върху менюто **Schemas** . Той ще покаже всички налични бази данни в MySQL сървъра за бази данни.



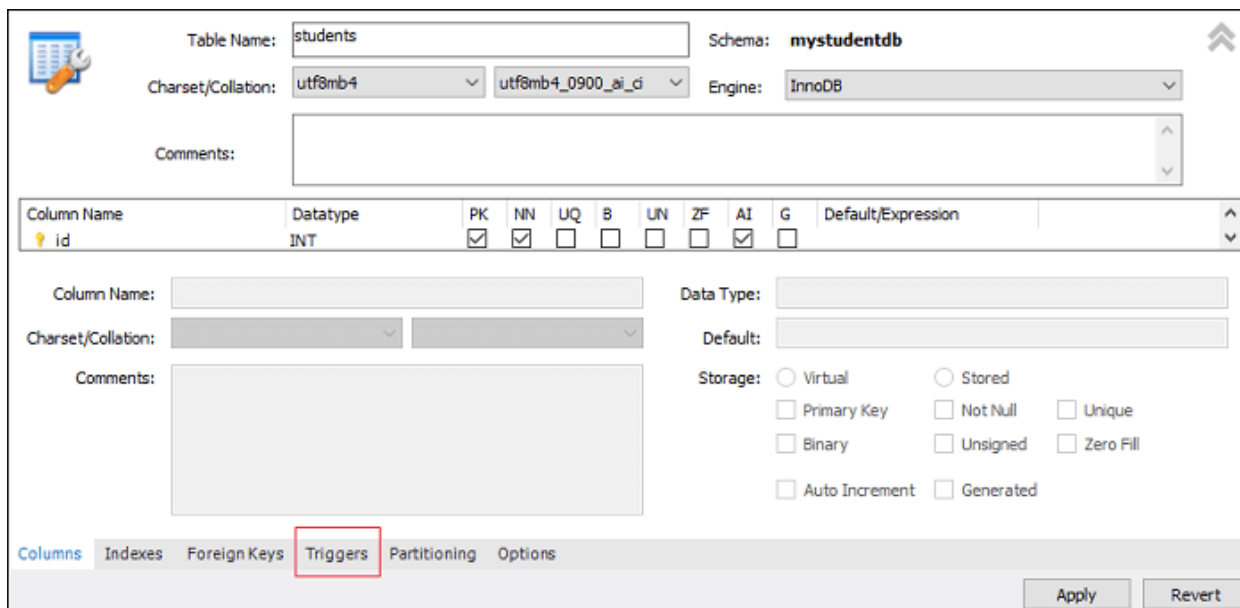
2. Изберете базата данни (например **mystudentdb**). След това кликнете два пъти върху избраната схема. Той показва **подменюто**, съдържащо таблици, изгледи, функции и съхранени процедури.



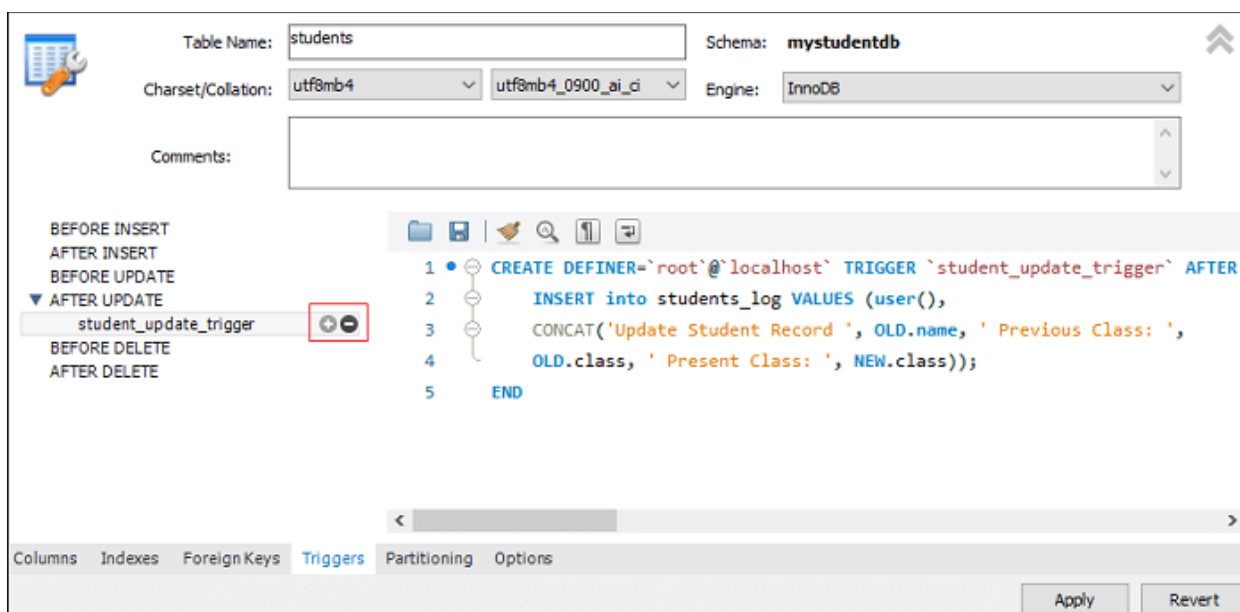
3. Разгънете подменюто **Tables** и изберете таблица, към която е свързан тригер. Отново разгънете избраната **Таблица -> Triggers**:



4. Сега щракнете с десния бутон върху избраната таблица и изберете опцията **Alter Table**, която дава екрана:



5. Сега щракнете върху раздела **Trigger**, показан в червеното правоъгълно поле. Ще забележите, че има **бутон с икона (+) и (-)** за добавяне или изтриване на тригер:



6. Сега, щракването върху **бутона (-)** ще премахне завинаги тригера, свързан с таблицата.

MySQL BEFORE INSERT TRIGGER

Insert Trigger в MySQL се извиква автоматично, когато се изпълнява операция за вмъкване. Как да създадем тригер преди вмъкване с неговия синтаксис?

Синтаксис

Създаване на тригер **BEFORE INSERT** в MySQL :

```
CREATE TRIGGER trigger_name
```

```
BEFORE INSERT
```

```
ON table_name FOR EACH ROW
```

```
Trigger_body ;
```

Параметърът на тригера **BEFORE INSERT** може да обясним така:

- Първо, ще посочим **trigger_name / името на тригера** , който искаме да създадем. То трябва да бъде уникално в рамките на схемата.
- Второ, ще посочим **trigger action time / времето за действие на тригера** , което трябва да бъде **BEFORE INSERT**. Този тригер ще бъде извикан, преди да се появят промени на всеки ред в таблицата.
- Трето, ще посочим **table_name / името на таблица** , към която е свързан тригерът. Трябва да се напише след ключовата дума **ON**. Ако не се посочи името на таблицата, тригерът няма да съществува.
- Накрая ще посочим оператора за изпълнение, когато тригерът е активиран.

Ако искаме да изпълним множество изрази, ще използваме блока **BEGIN END**, който съдържа набор от заявки за дефиниране на логиката за тригера.

Синтаксис:

```
DELIMITER $$
```

```
CREATE TRIGGER trigger_name BEFORE INSERT
```

```
ON table_name FOR EACH ROW
```

```
BEGIN
```

```
variable declarations
```

```
trigger code
```

```
END$$
```

```
DELIMITER ;
```

Ограничения

- Можем да имаме достъп и да променяме **НОВИТЕ** стойности само в тригер **BEFORE INSERT**.
- Нямаме достъп до **СТАРИТЕ**. Ако се опитаме да получим достъп до **СТАРИТЕ** стойности, ще получим грешка, защото **СТАРИ** стойности не съществуват.
- Не можем да създадем тригер **BEFORE INSERT** на **VIEW**.

Пример за тригер BEFORE INSERT

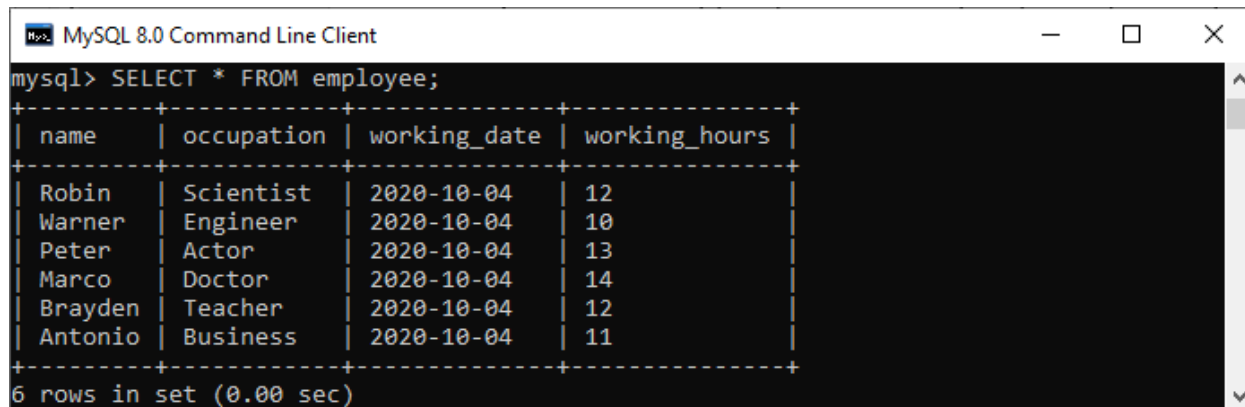
Как да създадем тригер **BEFORE INSERT** с помощта на оператора **CREATE TRIGGER** в MySQL

Да предположим, че сме създали таблица с име **employee** , както следва:

```
CREATE TABLE employee(
```

```
name varchar(45) NOT NULL,  
occupation varchar(35) NOT NULL,  
working_date date,  
working_hours varchar(10)  
);
```

След това ще вмъкнем някои записи в таблицата на служителите и след това ще изпълним SELECT, за да видим данните от таблицата, както следва:



```
mysql> SELECT * FROM employee;
```

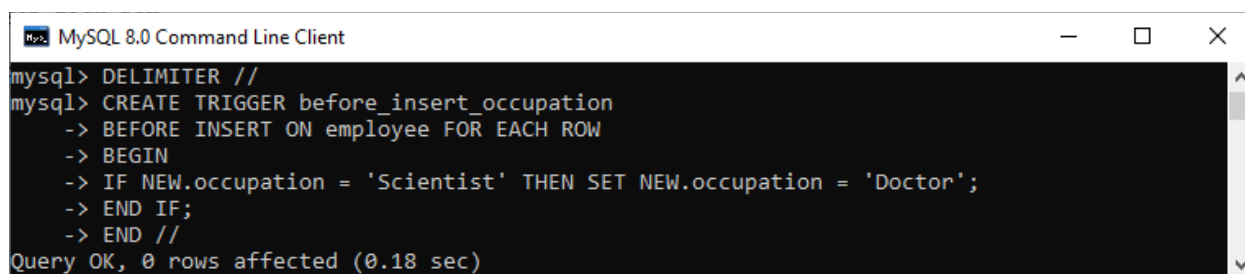
name	occupation	working_date	working_hours
Robin	Scientist	2020-10-04	12
Warner	Engineer	2020-10-04	10
Peter	Actor	2020-10-04	13
Marco	Doctor	2020-10-04	14
Brayden	Teacher	2020-10-04	12
Antonio	Business	2020-10-04	11

```
6 rows in set (0.00 sec)
```

След това ще използваме оператор **CREATE TRIGGER**, за да създадем тригер BEFORE INSERT. Този тригер се извиква автоматично и вмъква **occupation = 'Leader'**, ако някой се опита да вмъкне **occupation = 'Scientist'**.

```
DELIMITER //  
  
Create Trigger before_insert_occupation  
  
BEFORE INSERT ON employee FOR EACH ROW  
  
BEGIN  
  
IF NEW.occupation = 'Scientist' THEN SET NEW.occupation = 'Doctor';  
  
END IF;  
  
END //
```

Ако тригерът е създаден успешно, ще получим резултата, както следва:



```
mysql> DELIMITER //  
mysql> CREATE TRIGGER before_insert_occupation  
-> BEFORE INSERT ON employee FOR EACH ROW  
-> BEGIN  
-> IF NEW.occupation = 'Scientist' THEN SET NEW.occupation = 'Doctor';  
-> END IF;  
-> END //  
Query OK, 0 rows affected (0.18 sec)
```

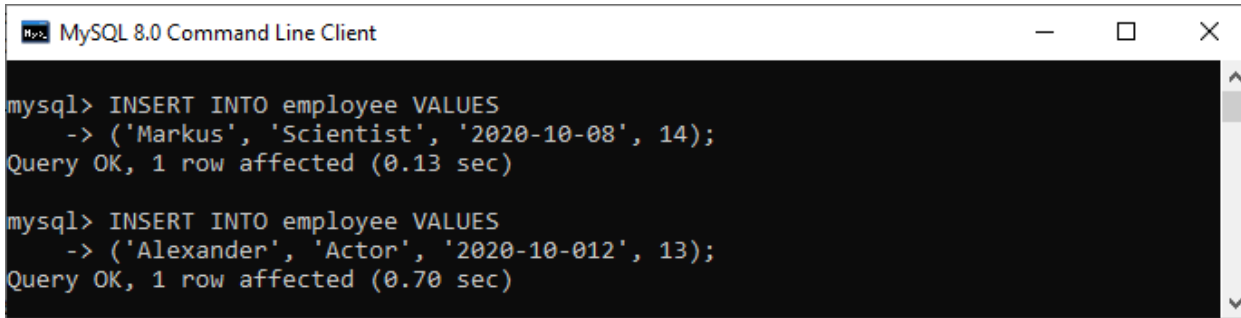
Как да извикам тригера BEFORE INSERT?

Можем да използваме следните изрази, за да извикаме създадения тригер:


```
INSERT INTO employee VALUES ('Markus', 'Scientist', '2020-10-08', 14);
```

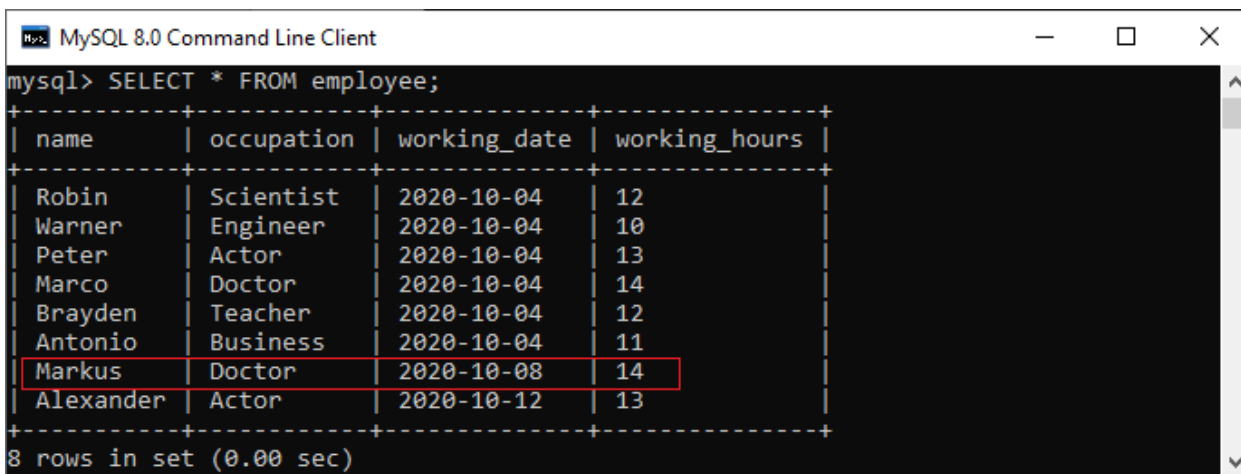
```
INSERT INTO employee VALUES ('Alexander', 'Actor', '2020-10-012', 13);
```

След изпълнение на горния оператор, ще получим:



```
mysql> INSERT INTO employee VALUES  
-> ('Markus', 'Scientist', '2020-10-08', 14);  
Query OK, 1 row affected (0.13 sec)  
  
mysql> INSERT INTO employee VALUES  
-> ('Alexander', 'Actor', '2020-10-012', 13);  
Query OK, 1 row affected (0.70 sec)
```

Изпълнете командата **SELECT**, за да проверите изхода:



```
mysql> SELECT * FROM employee;
```

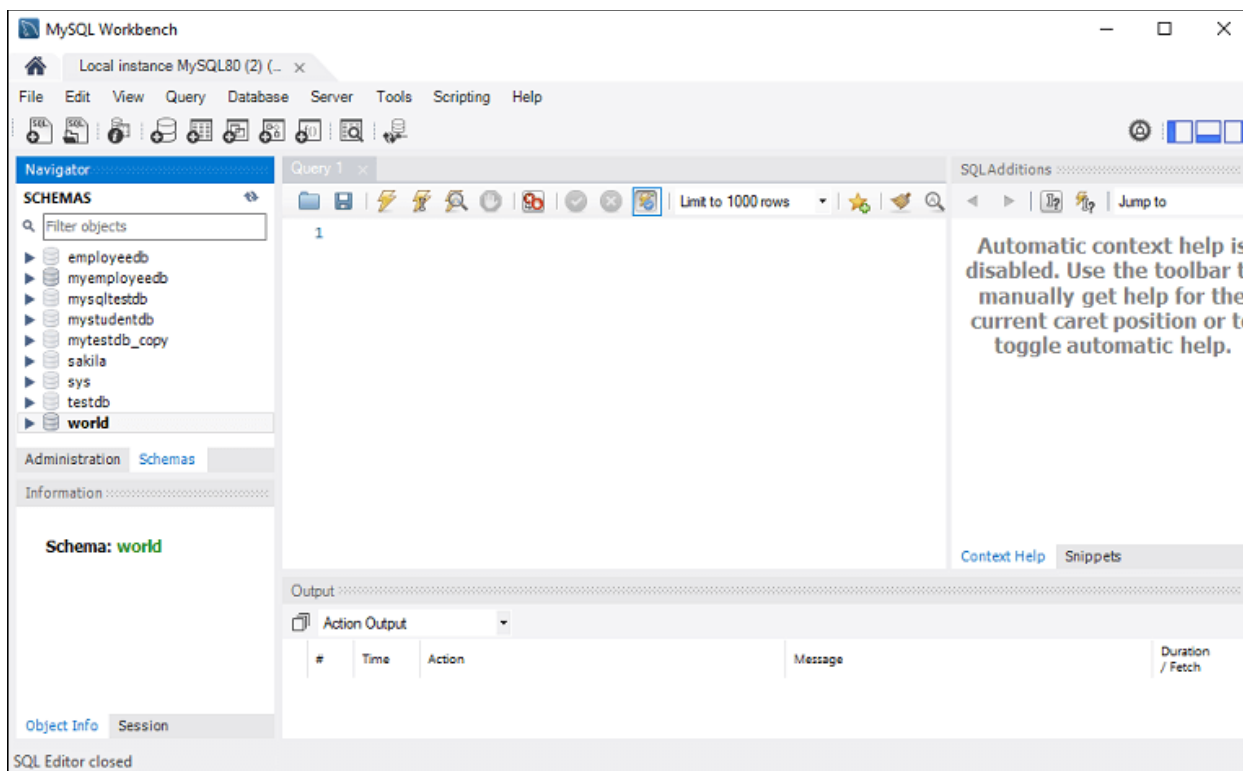
name	occupation	working_date	working_hours
Robin	Scientist	2020-10-04	12
Warner	Engineer	2020-10-04	10
Peter	Actor	2020-10-04	13
Marco	Doctor	2020-10-04	14
Brayden	Teacher	2020-10-04	12
Antonio	Business	2020-10-04	11
Markus	Doctor	2020-10-08	14
Alexander	Actor	2020-10-12	13

```
8 rows in set (0.00 sec)
```

В този изход можем да видим, че при вмъкване на стойностите на колоната за професия като 'Scientist', таблицата автоматично ще попълни стойността 'Doctor' чрез извикване на тригер.

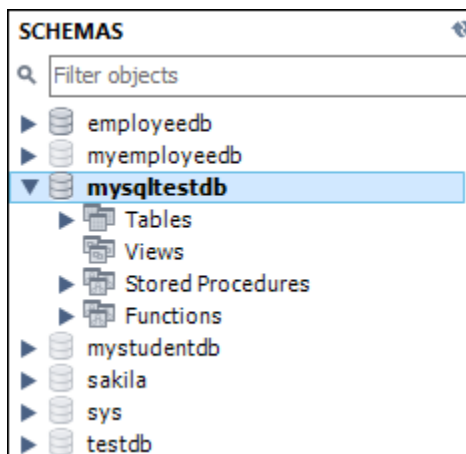
Как да създадете **BEFORE INSERT Trigger** в **MySQL workbench**?

За да създадем тригер преди вмъкване с помощта на този инструмент, първо трябва да стартираме MySQL Workbench и да влезем с потребителското име и парола. Ще получим екрана, както следва:

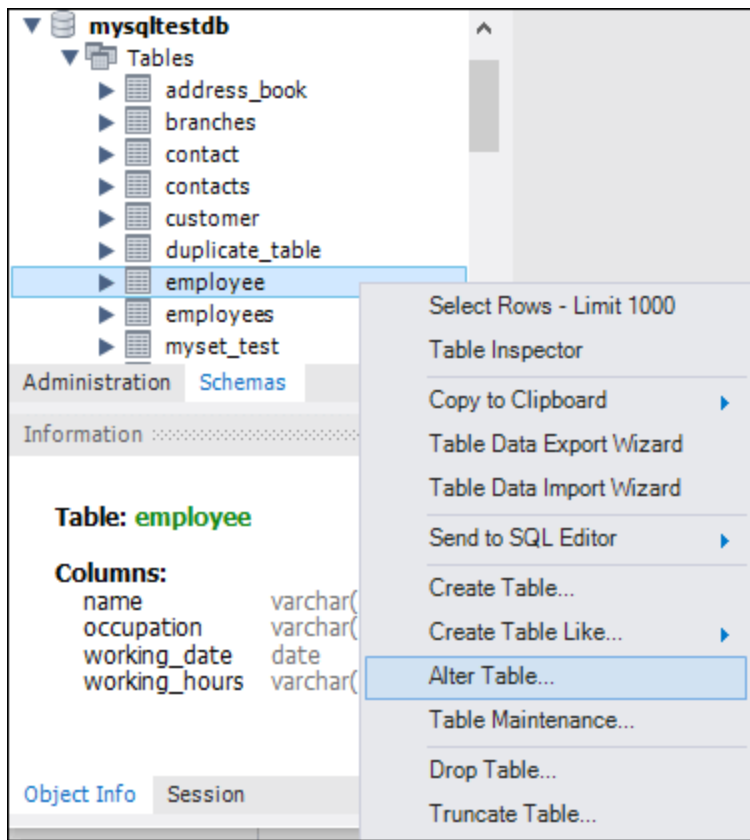


Сега направете следните стъпки за създаване на тригер **BEFORE INSERT**:

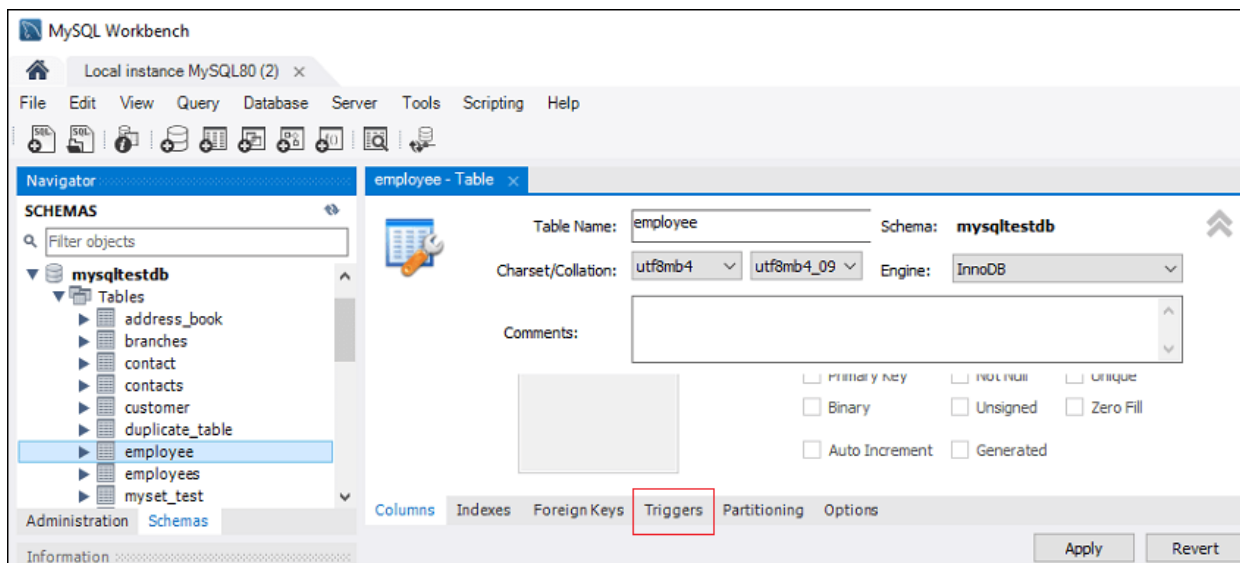
1. Отидете в раздела **Navigator** и щракнете върху **менюто Shemas** , което съдържа всички налични бази данни в MySQL сървъра.
2. Изберете базата данни (например **mysqltestdb**), щракнете двукратно върху нея. Ще покаже **подменюто**, съдържащо таблици, изгледи, функции и съхранени процедури.



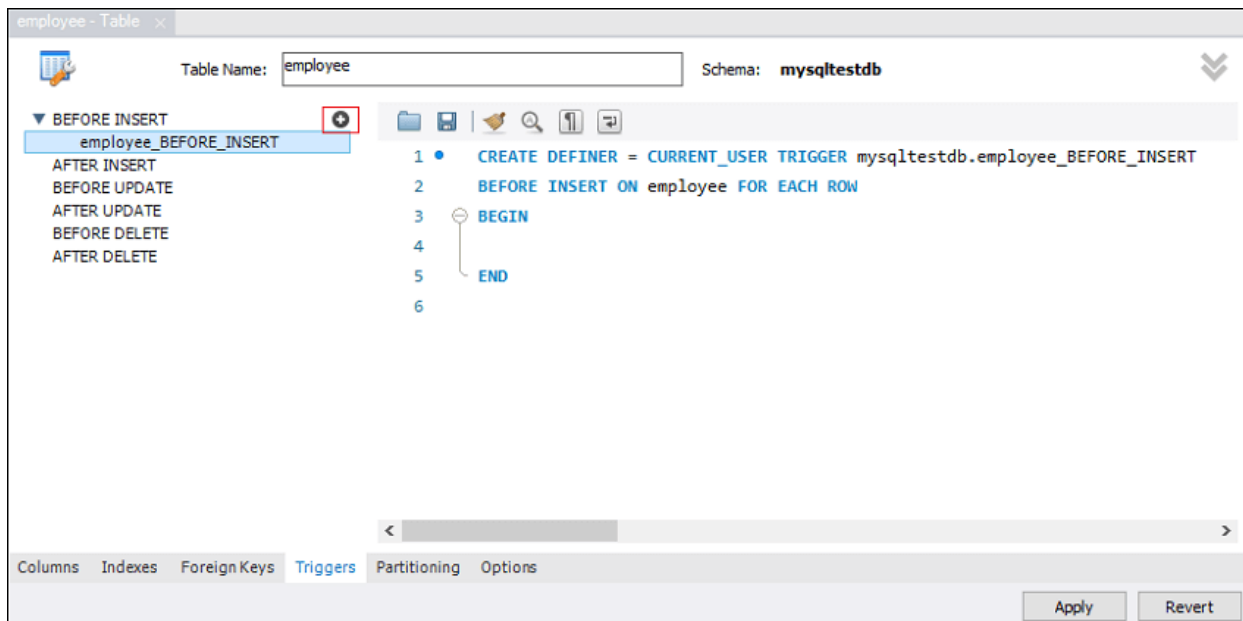
3. Разгънете **подменюто Tables** и изберете таблицата, за която искате да създадете тригер. След като изберете таблица, щракнете с десния бутон върху избраната таблица (например **Employee**) и след това щракнете върху опцията **Alter Table**.



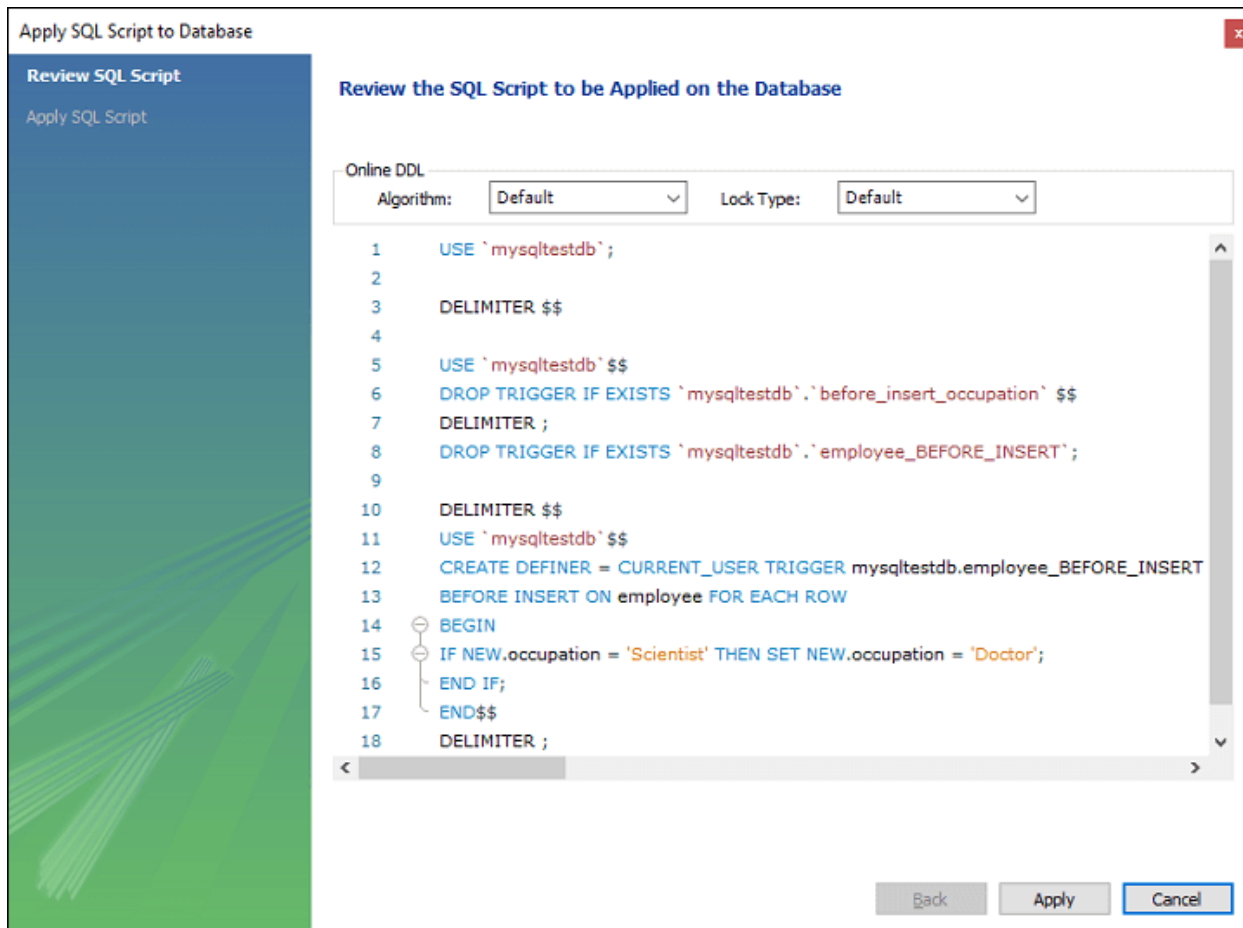
4. Щракването върху опцията за промяна на таблицата дава екрана:



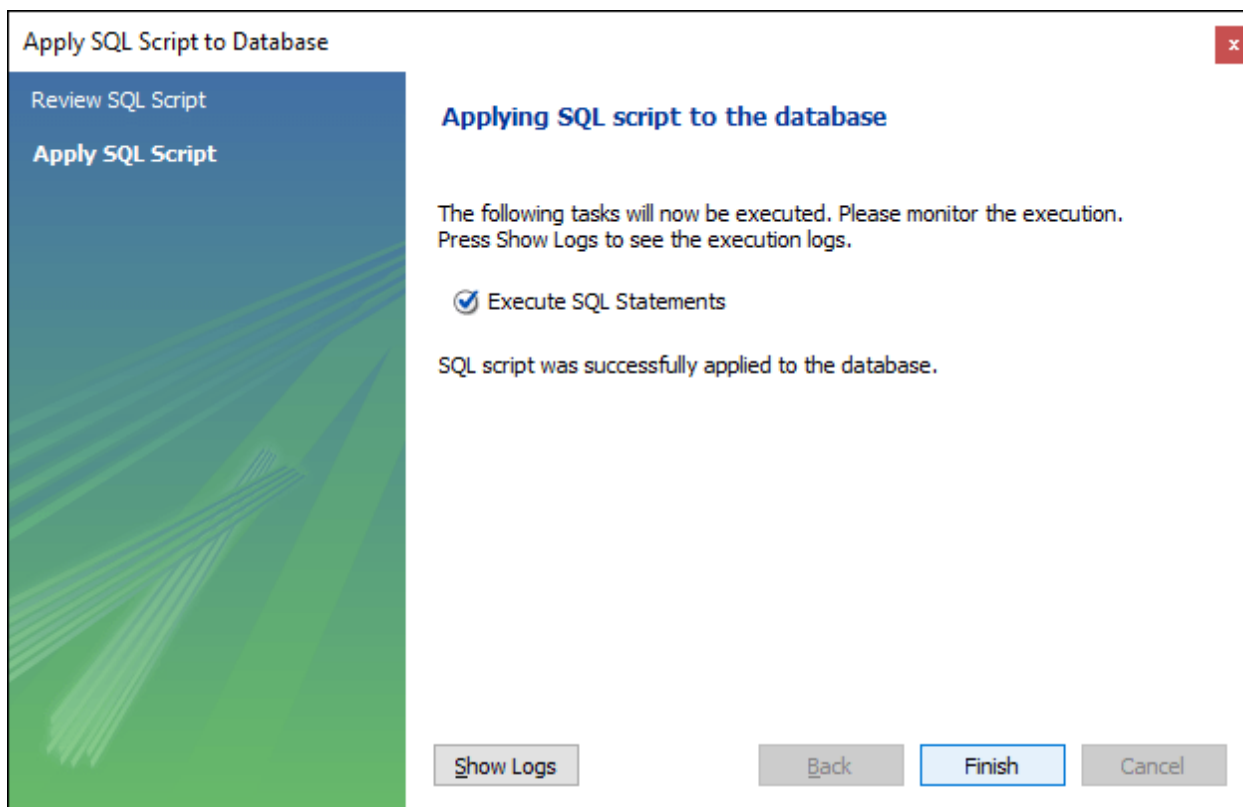
5. Сега щракнете върху **раздела Trigger** , показан в червено правоъгълно поле на предишния раздел, след което изберете **Timing/Event BEFORE INSERT**. Ще забележим, че има бутон с икона (+) за добавяне на тригер. Щраквайки върху този бутон, ще получим код по подразбиране на тригера въз основа на избора на Време/Събитие:



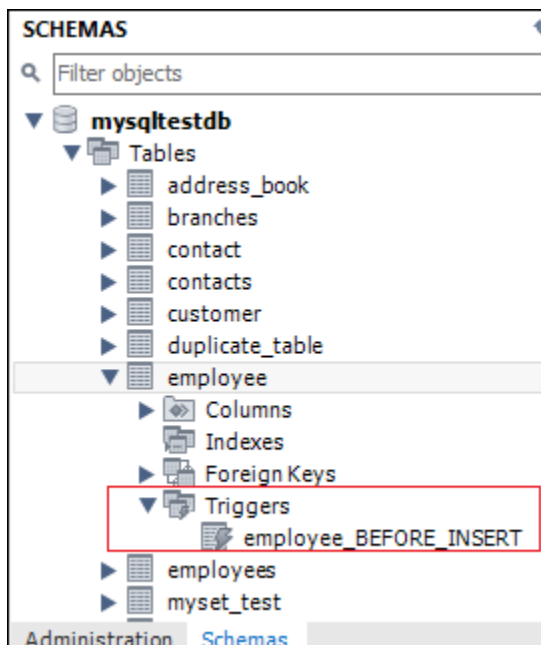
6. Сега попълнете кода на задействане, прегледайте ги още веднъж, ако не откриете грешка, щракнете върху **бутона Apply** .



7. След като щракнете върху **бутона Apply**, щракнете върху **бутона Finish** за завършване.



8. Ако влезем в менюто на схемата, ще видим задействане на **employee_BEFORE_INSERT** под таблицата на **employee**, както следва:



MySQL AFTER INSERT Trigger

INSERT Trigger в MySQL се извиква автоматично, когато в таблицата възникне събитие за вмъкване. Как да създадем тригер след вмъкване със съответния синтаксис.

Синтаксис

Създаване на тригер **AFTER INSERT** в MySQL :

```
CREATE TRIGGER trigger_name  
AFTER INSERT  
ON table_name FOR EACH ROW  
trigger_body ;
```

Параметри на тригера **AFTER INSERT**:

- Първо, ще посочим **trigger_name / името на тригера** , който искаме да създадем. То трябва да бъде уникално в рамките на схемата.
- Второ, ще посочим **trigger action time / времето за действие на тригера** , което трябва да бъде клауза **AFTER INSERT**, за да извика тригера.
- Трето, ще посочим **name of a table / името на таблица** , към която е свързан тригерът. Трябва да се напише след ключовата дума **ON**. Ако не посочим името на таблицата, тригерът няма да съществува.
- Накрая ще посочим **trigger body / тялото на тригера** , което съдържа един или повече изрази за изпълнение, когато тригерът е активиран.

Ако искаме да изпълним множество изрази, ще използваме блока **BEGIN END** , който съдържа набор от SQL заявки, за да дефинираме логиката за тригера. Вижте синтаксиса:

```
DELIMITER $$  
  
CREATE TRIGGER trigger_name AFTER INSERT  
ON table_name FOR EACH ROW  
BEGIN  
    variable declarations  
  
    trigger code  
  
END$$  
DELIMITER ;
```

Ограничения

- Имаме достъп до **NEW** стойности, но **не можем да ги променим** в тригер **AFTER INSERT**.
- Нямаме достъп до **OLD** стойности. Ако се опитаме да получим достъп до **OLD** стойностите, ще получим грешка, тъй като няма **OLD** в **INSERT** тригера.
- Не можем да създадем тригера **AFTER INSERT** на **VIEW**.

Пример за тригер AFTER INSERT

Как да създадем тригер **AFTER INSERT** с помощта на оператора **CREATE TRIGGER** в MySQL.

Създаваме таблица с име " **student_info** ", както следва:

```
CREATE TABLE student_info (
```

```

stud_id int NOT NULL,

stud_code varchar(15) DEFAULT NULL,

stud_name varchar(35) DEFAULT NULL,

subject varchar(25) DEFAULT NULL,

marks int DEFAULT NULL,

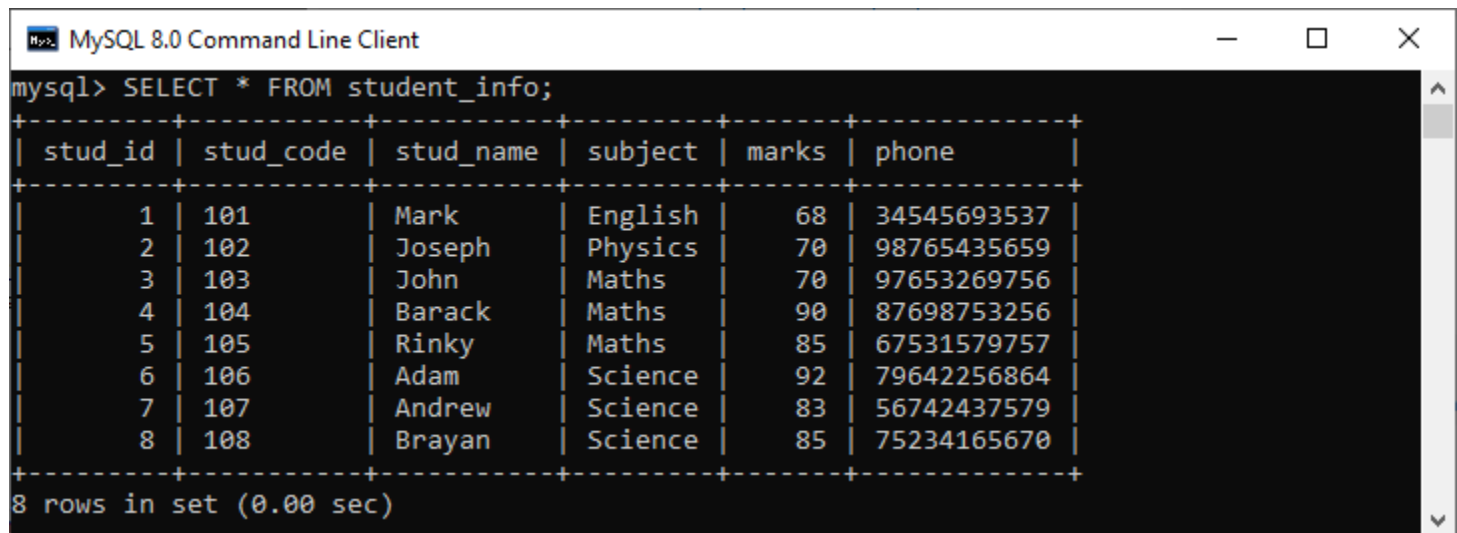
phone varchar(15) DEFAULT NULL,

PRIMARY KEY (stud_id)

)

```

След това ще вмъкнем някои записи в тази таблица и след това ще изпълним `SELECT`, за да видим данните от таблицата:



```

mysql> SELECT * FROM student_info;
+-----+-----+-----+-----+-----+-----+
| stud_id | stud_code | stud_name | subject | marks | phone |
+-----+-----+-----+-----+-----+-----+
| 1 | 101 | Mark | English | 68 | 34545693537 |
| 2 | 102 | Joseph | Physics | 70 | 98765435659 |
| 3 | 103 | John | Maths | 70 | 97653269756 |
| 4 | 104 | Barack | Maths | 90 | 87698753256 |
| 5 | 105 | Rinky | Maths | 85 | 67531579757 |
| 6 | 106 | Adam | Science | 92 | 79642256864 |
| 7 | 107 | Andrew | Science | 83 | 56742437579 |
| 8 | 108 | Brayan | Science | 85 | 75234165670 |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

```

Ще създадем нова таблица с име "**student_detail**", както следва:

```

CREATE TABLE student_detail (

stud_id int NOT NULL,

stud_code varchar(15) DEFAULT NULL,

stud_name varchar(35) DEFAULT NULL,

subject varchar(25) DEFAULT NULL,

marks int DEFAULT NULL,

phone varchar(15) DEFAULT NULL,

Lastinserted Time,

PRIMARY KEY (stud_id)

);

```

След това ще използваме оператор **CREATE TRIGGER**, за да създадем тригер **after_insert_details** в таблицата **student_info** . Този тригер ще се задейства след извършване на операция за вмъкване в таблицата.

```
DELIMITER //

Create Trigger after_insert_details

AFTER INSERT ON student_info FOR EACH ROW

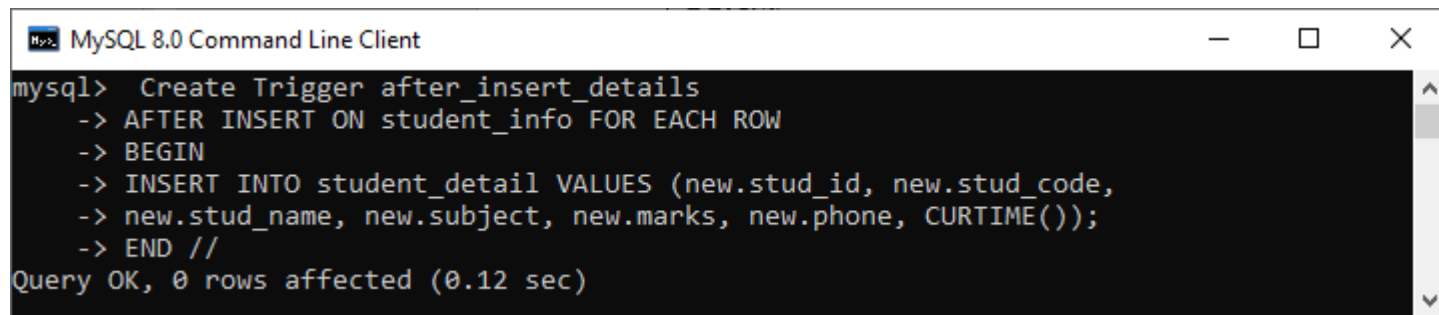
BEGIN

INSERT INTO student_detail VALUES (new.stud_id, new.stud_code,

new.stud_name, new.subject, new.marks, new.phone, CURTIME());

END //
```

Ако тригерът е създаден успешно, ще получим резултата:



```
MySQL 8.0 Command Line Client

mysql> Create Trigger after_insert_details
-> AFTER INSERT ON student_info FOR EACH ROW
-> BEGIN
-> INSERT INTO student_detail VALUES (new.stud_id, new.stud_code,
-> new.stud_name, new.subject, new.marks, new.phone, CURTIME());
-> END //
Query OK, 0 rows affected (0.12 sec)
```

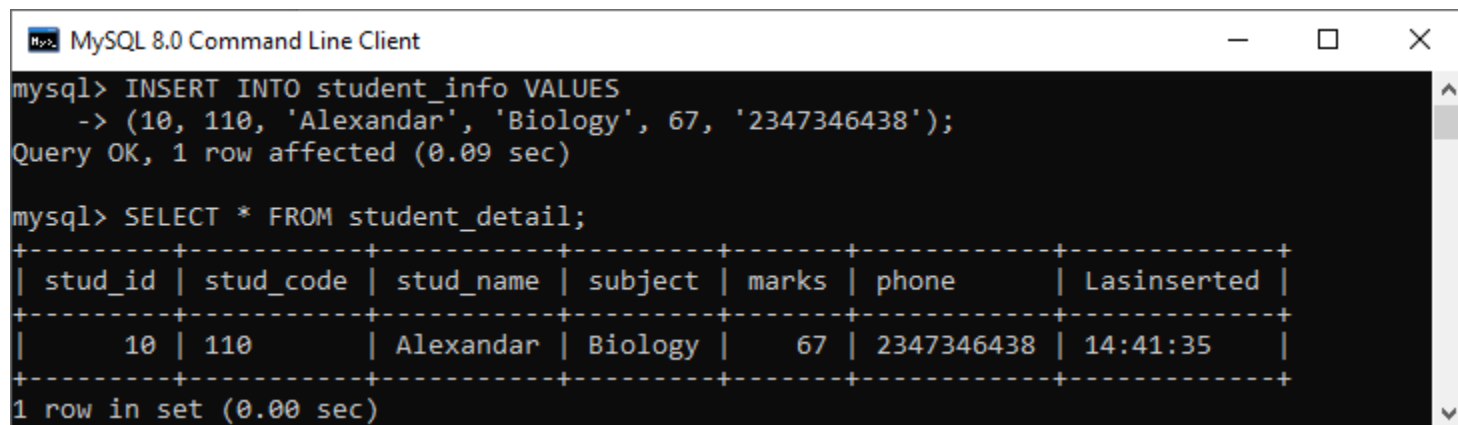
Как да извикам тригера **AFTER INSERT**?

Можем да използваме следните изрази, за да извикаме създадения тригер:

```
INSERT INTO student_info VALUES (10, 110, 'Alexandar', 'Biology', 67, '2347346438');
```

Таблицата, която е била модифицирана след изпълнението на заявката за актуализиране, е **student_detail**. Можем да го проверим, като използваме командата **SELECT**, както следва:

```
SELECT * FROM student_detail;
```



```
MySQL 8.0 Command Line Client

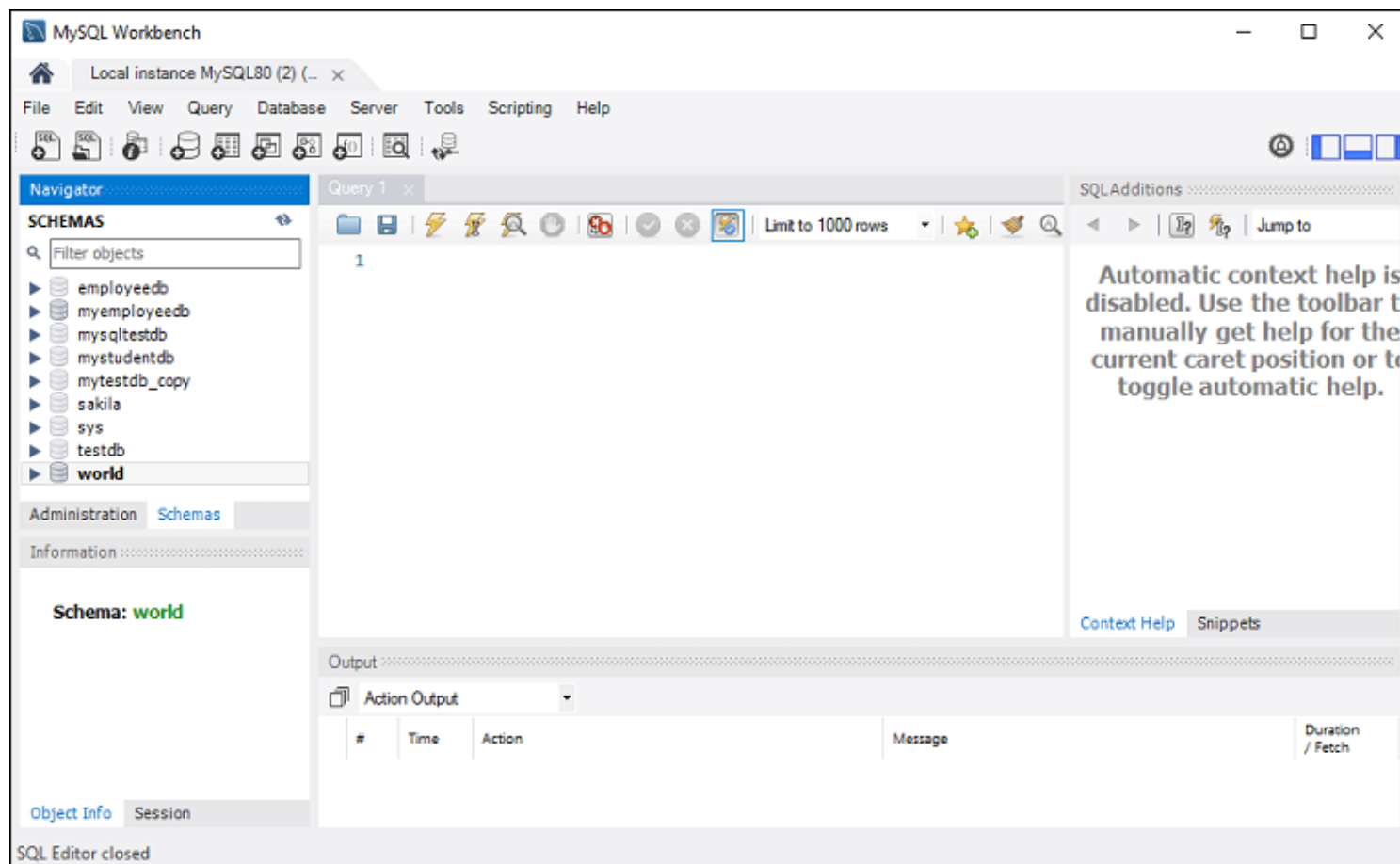
mysql> INSERT INTO student_info VALUES
-> (10, 110, 'Alexandar', 'Biology', 67, '2347346438');
Query OK, 1 row affected (0.09 sec)

mysql> SELECT * FROM student_detail;
+-----+-----+-----+-----+-----+-----+-----+
| stud_id | stud_code | stud_name | subject | marks | phone       | Lasinserted |
+-----+-----+-----+-----+-----+-----+-----+
|      10 |      110 | Alexandar | Biology |     67 | 2347346438 | 14:41:35    |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```


В този изход можем да видим, че при вмъкване на стойности в таблицата **student_info**, таблицата **student_detail** автоматично ще попълни записите чрез извикване на тригер.

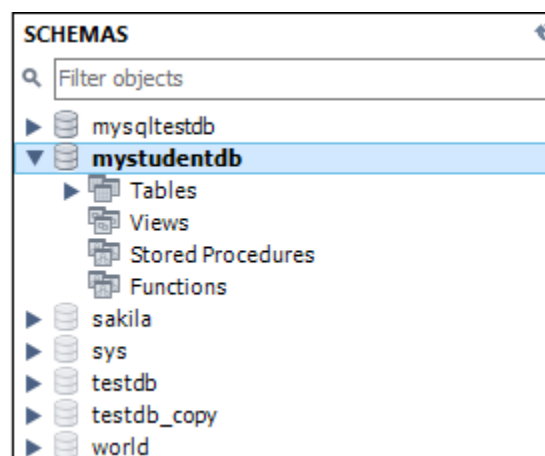
Как да създадете **AFTER INSERT Trigger** в **MySQL workbench**?

За да създадем тригер след вмъкване с помощта на този инструмент, първо трябва да **стартираме MySQL Workbench** и да влезем с потребителското име и парола:

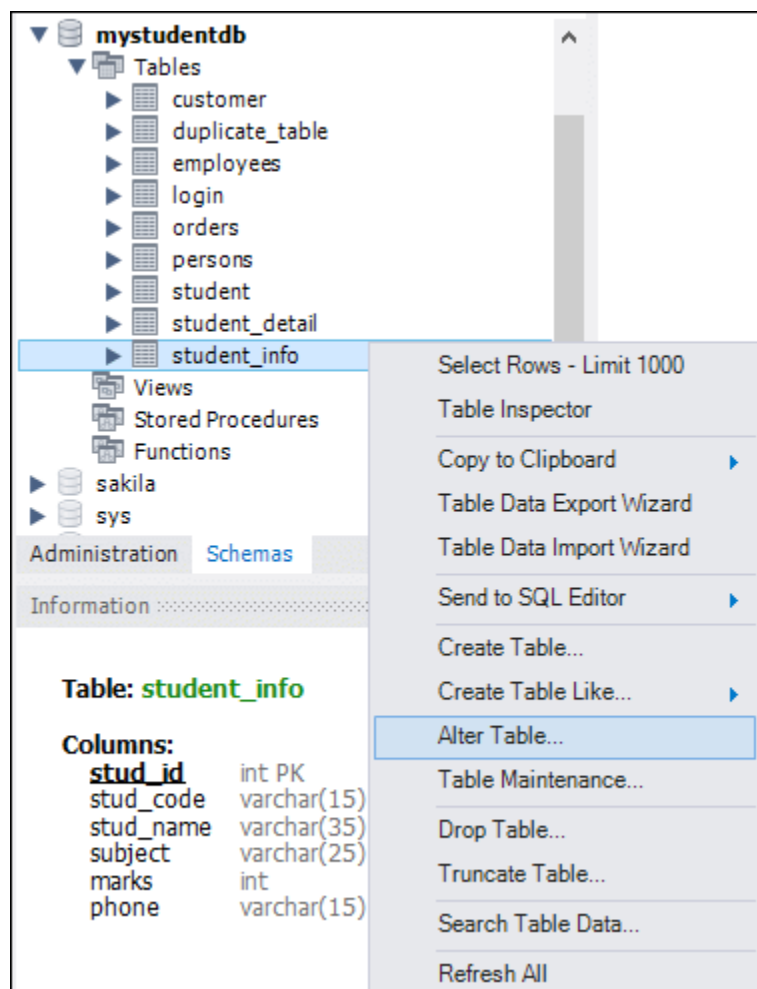


Сега направете следните стъпки за създаване на тригер **AFTER INSERT**:

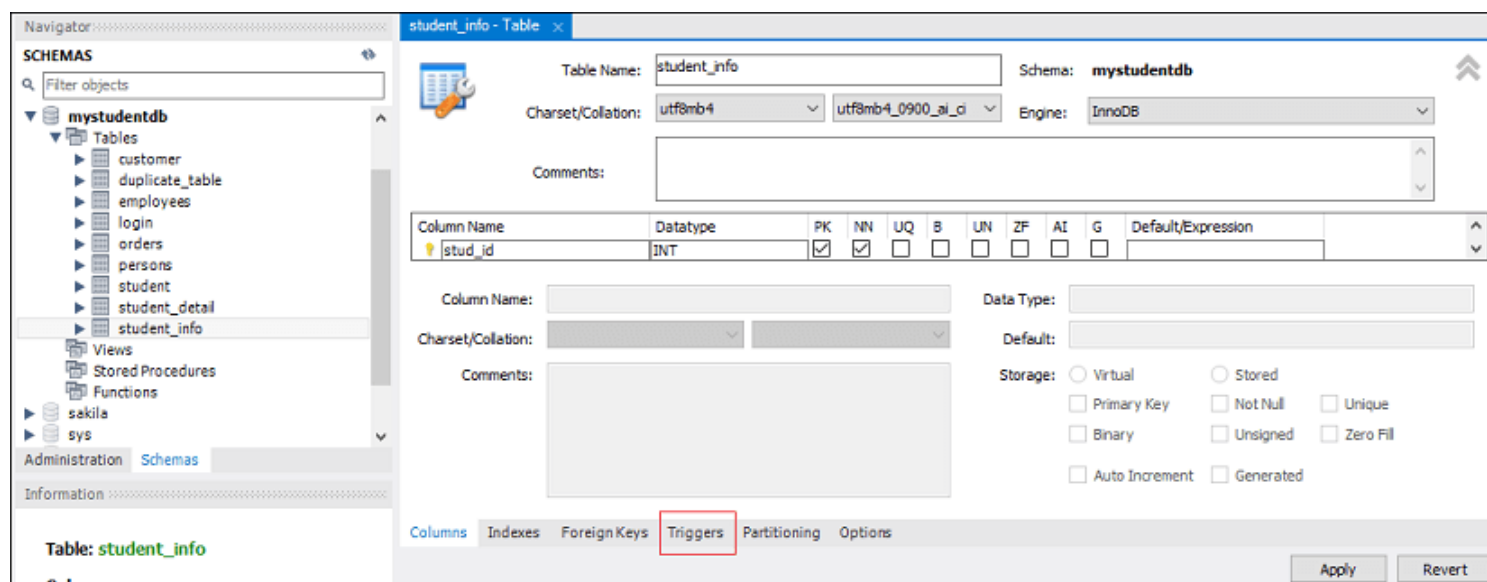
1. Отидете в раздела **Navigator** и щракнете върху **менюто Shemas** , което съдържа всички налични бази данни в MySQL сървъра.
2. Изберете базата данни (например **mystudentdb**), щракнете двукратно върху нея, което показва **подменюто** , съдържащо таблици, изгледи, функции и съхранени процедури.



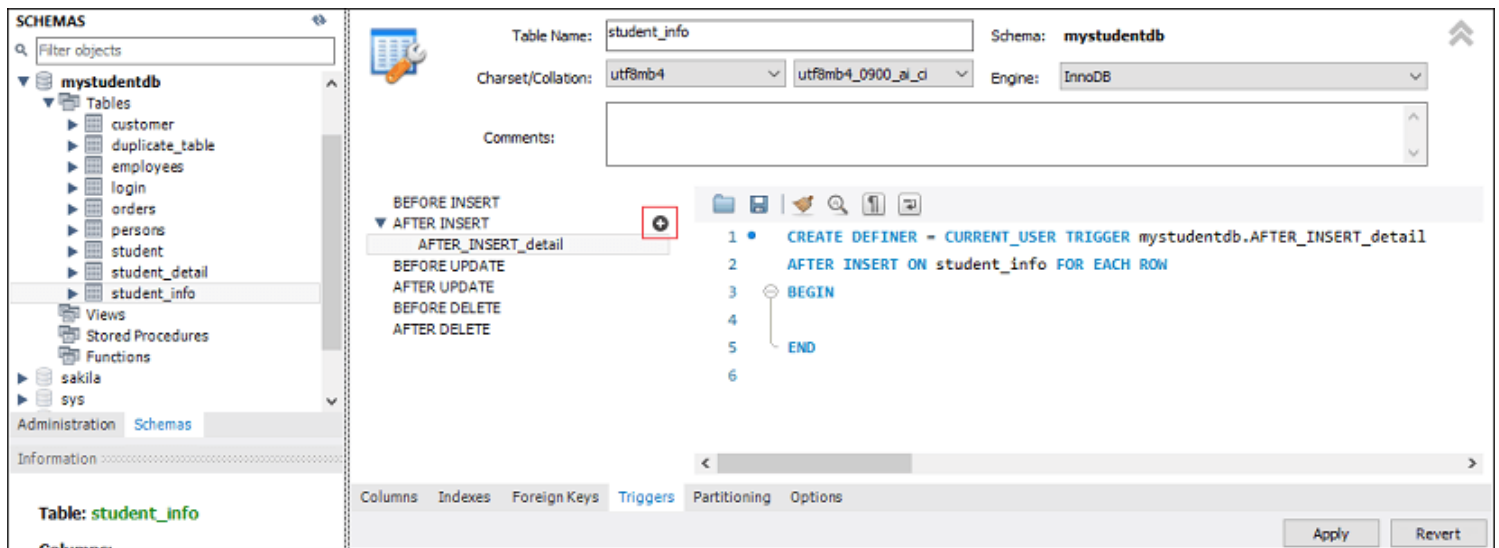
3. Разгънете **подменюто Tables** и изберете таблицата, за която искате да създадете тригер. След като изберете таблица, щракнете с десния бутон върху избраната таблица (например **mystudentdb**) и след това щракнете върху опцията **Alter Table**.



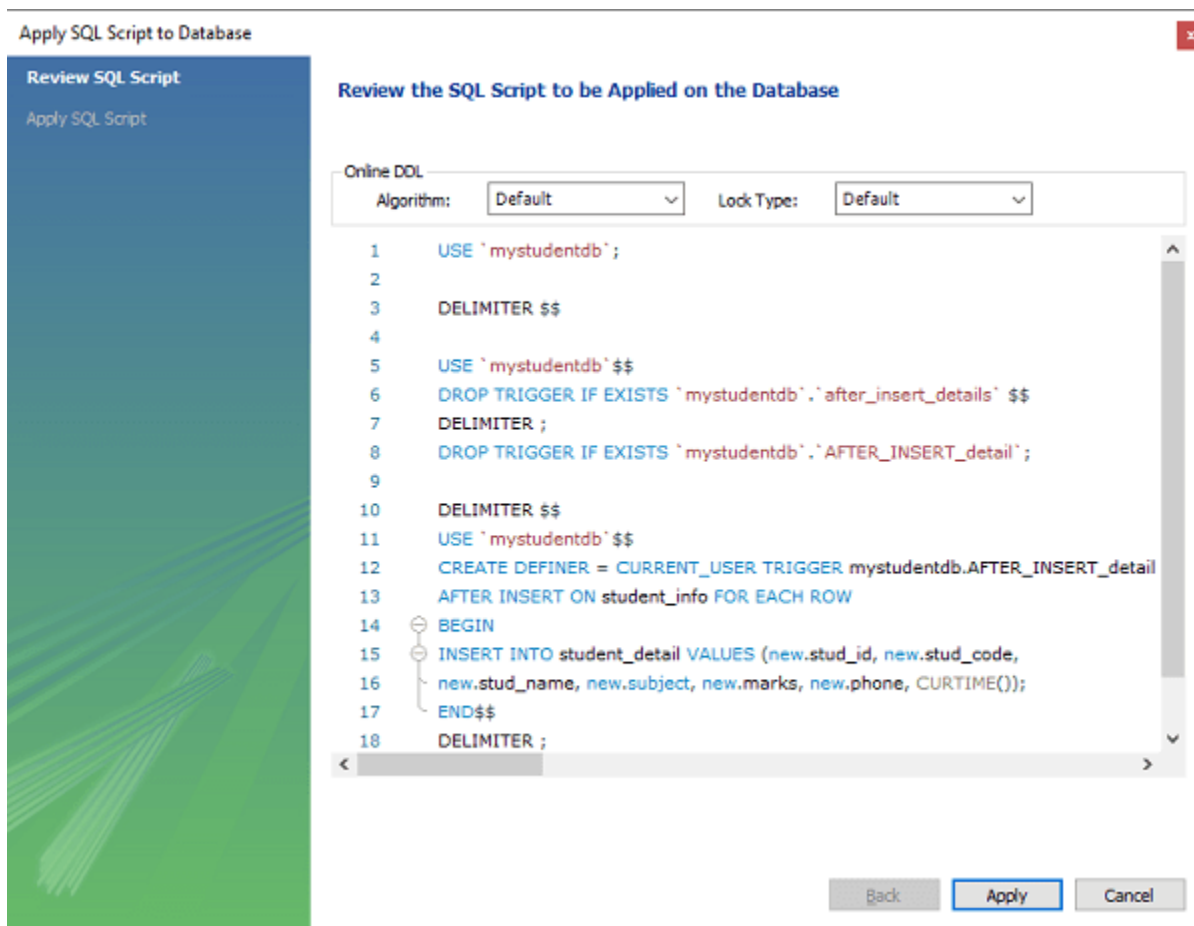
4. Щракването върху опцията за промяна на таблицата / **Alter Table** дава екрана, както е показано по-долу:



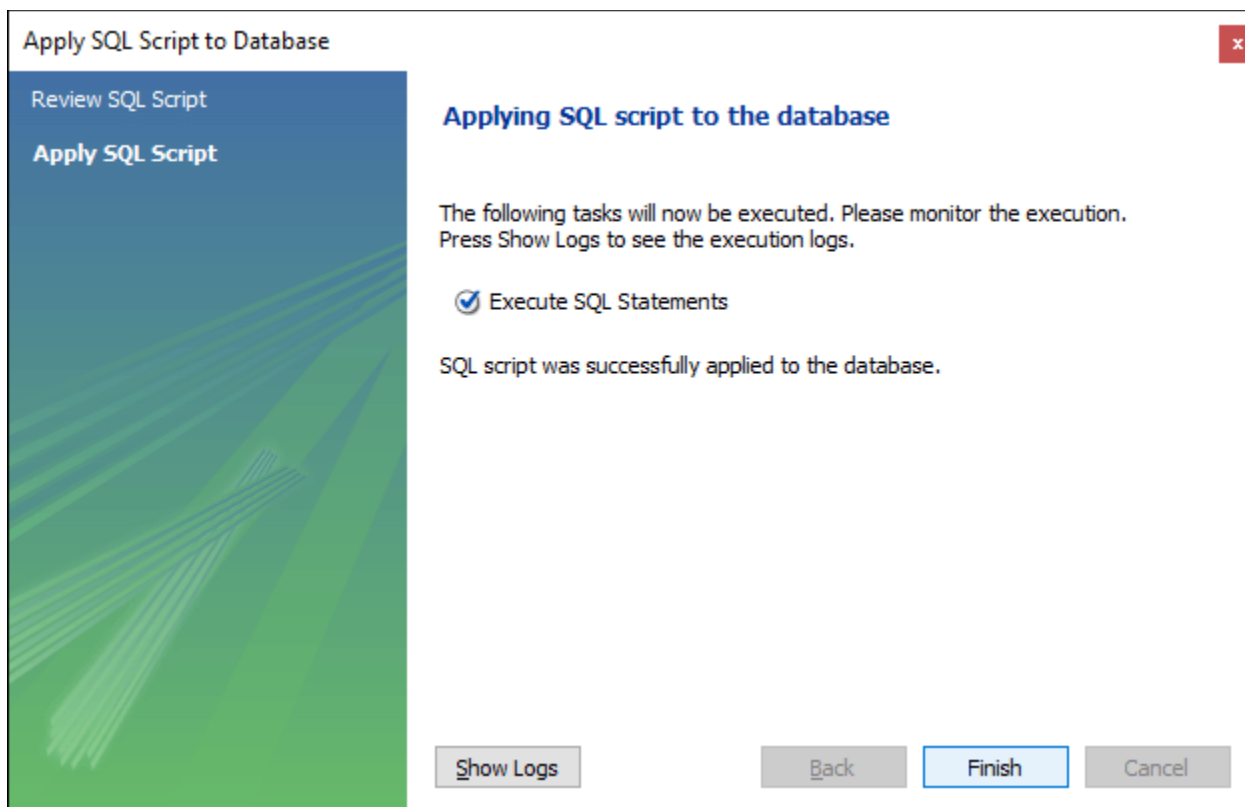
5. Сега щракнете върху **раздела Trigger**, показан в червеното правоъгълно поле на предишния раздел, след което изберете Timing/Event **AFTER INSERT**. Има бутон с икона (+) за добавяне на тригер. Щраквайки върху този бутон, ще получим код по подразбиране на тригера въз основа на избора на Време/Събитие:



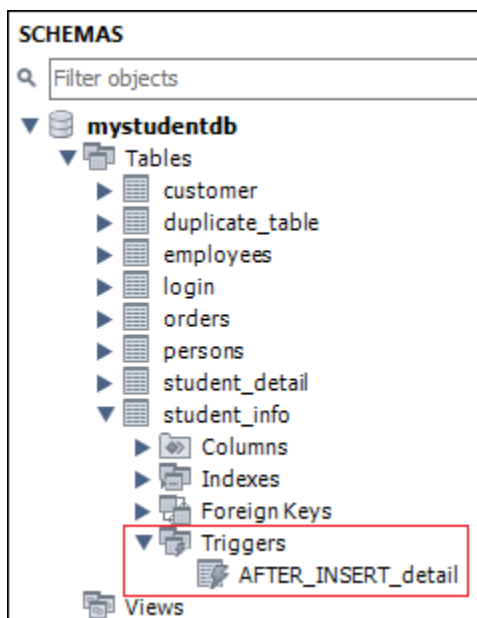
6. Сега попълнете тригерния код, прегледайте ги още веднъж и ако не бъде намерена грешка, щракнете върху бутона **Apply**.



7. След като щракнете върху бутона **Apply**, щракнете върху бутона **Finish** за завършване.



8. Ако погледнем менюто на схемата, можем да видим тригера **AFTER_INSERT_detail** под таблицата **student_info** , както следва:



MySQL BEFORE UPDATE Trigger

BEFORE UPDATE тригерът в MySQL се извиква автоматично, когато се задейства операция за актуализиране на таблицата, свързана с тригера. Как да създадем тригер преди актуализиране със съответния синтаксис.

Синтаксис

Създаване на тригер BEFORE UPDATE в MySQL:

```
CREATE TRIGGER trigger_name  
BEFORE UPDATE  
ON table_name FOR EACH ROW  
trigger_body ;
```

Параметри на тригера BEFORE UPDATE:

- Първо, ще посочим **trigger name / името на тригера** , който искаме да създадем. Тя трябва да бъде уникално в рамките на схемата.
- Второ, ще посочим **trigger action time / времето за действие на тригера** , което трябва да бъде BEFORE UPDATE. Този тригер ще бъде извикан преди всеки ред от промени да се случи в таблицата.
- Трето, ще посочим **name of a table / името на таблица**, към която е свързан тригерът. Трябва да се напише след ключовата дума **ON** . Ако не посочи името на таблицата, тригерът няма да съществува.
- Накрая ще посочим **trigger body / тялото на тригера** , което съдържа оператор за изпълнение, когато тригерът е активиран.

Ако искаме да изпълним множество изрази, ще използваме блока **BEGIN END** , който съдържа набор от заявки за дефиниране на логиката за тригера. Синтаксис:

```
DELIMITER $$  
  
CREATE TRIGGER trigger_name BEFORE UPDATE  
  
ON table_name FOR EACH ROW  
  
BEGIN  
  
    variable declarations  
  
    trigger code  
  
END$$  
  
DELIMITER;
```

Ограничения

- Не можем да актуализираме OLD стойности в тригер BEFORE UPDATE.
- Можем да променим NEW стойности.
- Не можем да създадем задействане BEFORE UPDATE на VIEW.

Пример за тригер BEFORE UPDATE

Как да създадем тригер **BEFORE UPDATE** с помощта на оператора **CREATE TRIGGER** в MySQL.

Създаваме таблица с име **sales_info** , както следва:

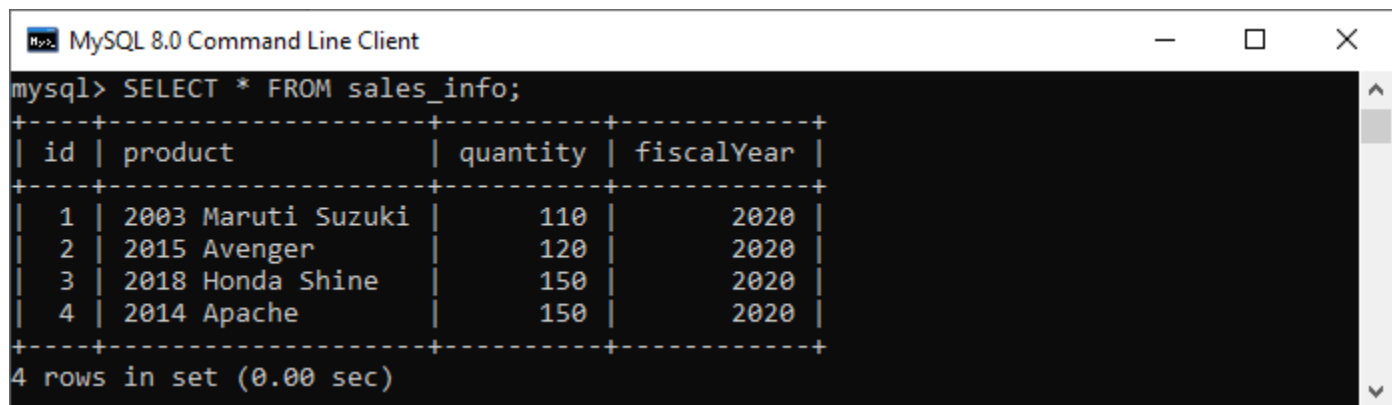
```
CREATE TABLE sales_info (  
  
    id INT AUTO_INCREMENT,
```

```
product VARCHAR(100) NOT NULL,  
quantity INT NOT NULL DEFAULT 0,  
fiscalYear SMALLINT NOT NULL,  
CHECK(fiscalYear BETWEEN 2000 and 2050),  
CHECK (quantity >=0),  
UNIQUE(product, fiscalYear),  
PRIMARY KEY(id)  
);
```

След това ще вмъкнем някои записи в таблицата **sales_info**, както следва:

```
INSERT INTO sales_info(product, quantity, fiscalYear)  
VALUES  
('2003 Maruti Suzuki',110, 2020),  
('2015 Avenger', 120,2020),  
('2018 Honda Shine', 150,2020),  
('2014 Apache', 150,2020);
```

След това изпълнете командата **SELECT** , за да видите данните от таблицата:



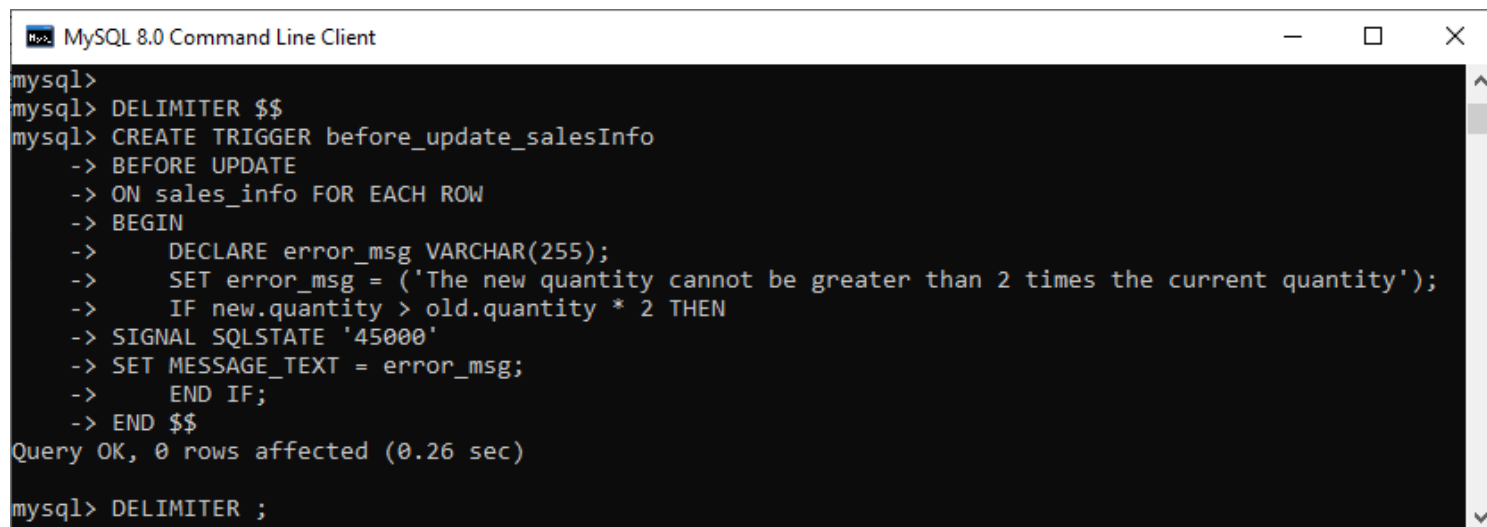
```
MySQL 8.0 Command Line Client  
mysql> SELECT * FROM sales_info;  
+----+-----+-----+-----+  
| id | product          | quantity | fiscalYear |  
+----+-----+-----+-----+  
| 1  | 2003 Maruti Suzuki | 110      | 2020      |  
| 2  | 2015 Avenger      | 120      | 2020      |  
| 3  | 2018 Honda Shine  | 150      | 2020      |  
| 4  | 2014 Apache       | 150      | 2020      |  
+----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

След това ще използваме оператор **CREATE TRIGGER** , за да създадем тригер **BEFORE UPDATE**. Този тригер се извиква автоматично, преди да възникне събитие за актуализиране в таблицата.

```
DELIMITER $$  
CREATE TRIGGER before_update_salesInfo  
BEFORE UPDATE  
ON sales_info FOR EACH ROW  
BEGIN  
  DECLARE error_msg VARCHAR(255);  
  SET error_msg = ('The new quantity cannot be greater than 2 times the current quantity');  
  IF new.quantity > old.quantity * 2 THEN
```

```
SIGNAL SQLSTATE '45000'  
SET MESSAGE_TEXT = error_msg;  
END IF;  
END $$  
DELIMITER ;
```

Ако тригерът е създаден успешно, ще получим резултата, както следва:



```
mysql>  
mysql> DELIMITER $$  
mysql> CREATE TRIGGER before_update_salesInfo  
-> BEFORE UPDATE  
-> ON sales_info FOR EACH ROW  
-> BEGIN  
->     DECLARE error_msg VARCHAR(255);  
->     SET error_msg = ('The new quantity cannot be greater than 2 times the current quantity');  
->     IF new.quantity > old.quantity * 2 THEN  
->     SIGNAL SQLSTATE '45000'  
->     SET MESSAGE_TEXT = error_msg;  
->     END IF;  
-> END $$  
Query OK, 0 rows affected (0.26 sec)  
  
mysql> DELIMITER ;
```

Тригерът извежда съобщение за грешка и спира актуализирането, ако актуализираме стойността в колоната за количество до нова стойност, два пъти по-голяма от текущата стойност.

Нека разгледаме подробно създадения тригер:

Първо сме посочили името на тригера като **before_update_salesInfo** в клаузата **CREATE TRIGGER**.

Второ, посочваме задействащото събитие и след това името на таблицата, към която е свързан задействащият елемент.

Трето, декларирали сме променлива и сме задали нейната стойност.

Четвърто, уточнихме тялото на тригера, което проверява дали новата стойност е два пъти по-голяма от старата стойност и след това предизвиква грешка.

Как да извикам тригера BEFORE UPDATE?

Първо, можем да използваме следните изрази, които актуализират количеството на реда, чийто id = 2:

```
UPDATE sales_info SET quantity = 125 WHERE id = 2;
```

Това работи добре, защото не нарушава правилото. След това ще изпълним инструкциите по-долу, които актуализират количеството като 600 на реда, чийто id = 2

```
UPDATE sales_info SET quantity = 600 WHERE id = 2;
```

Това ще даде грешка, защото нарушава правилото. Вижте изхода по-долу.

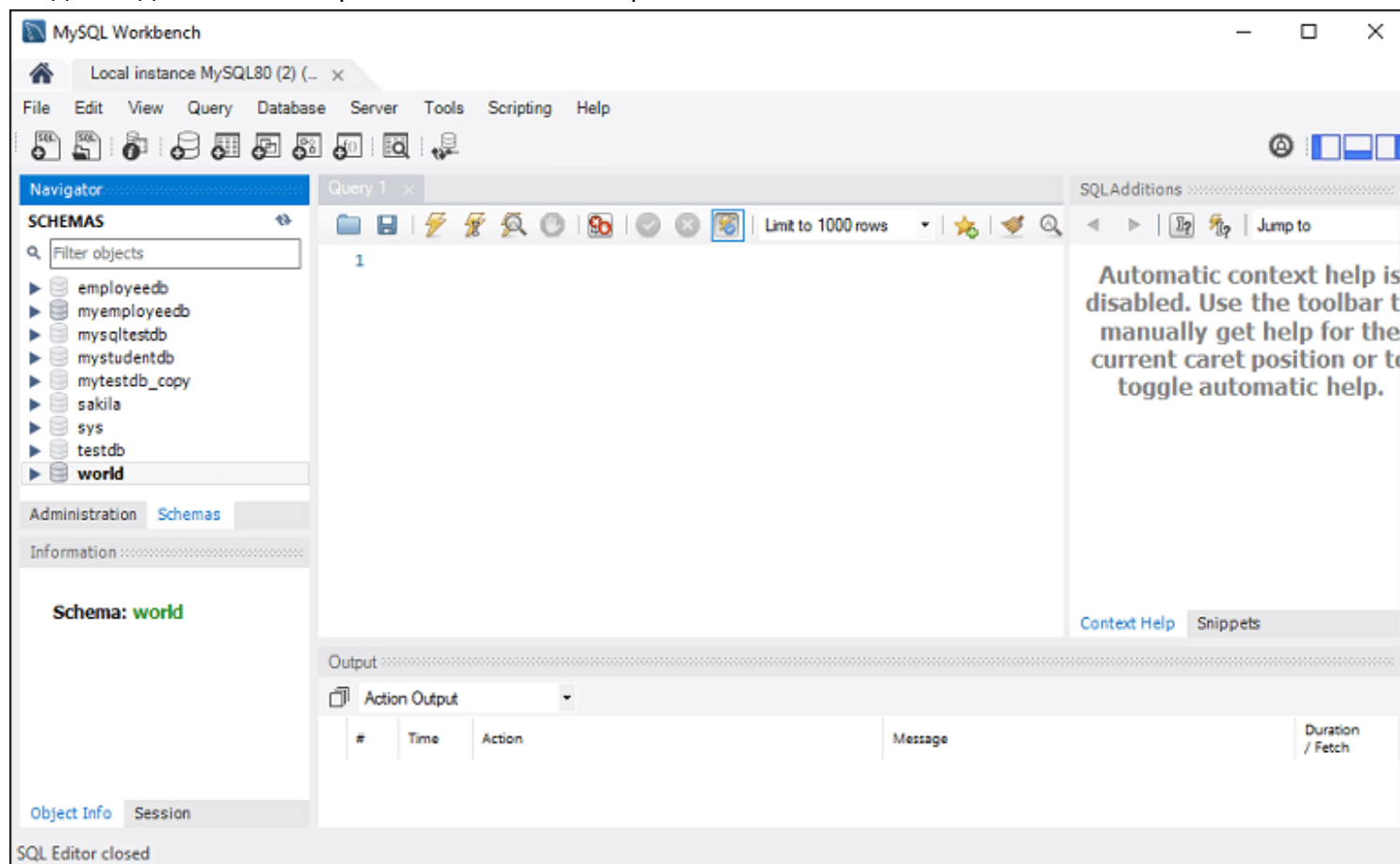
```
MySQL 8.0 Command Line Client

mysql> DELIMITER ;
mysql> UPDATE sales_info SET quantity = 125 WHERE id = 2;
Query OK, 1 row affected (0.08 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE sales_info SET quantity = 600 WHERE id = 2;
ERROR 1644 (45000): The new quantity cannot be greater than 2 times the current quantity
```

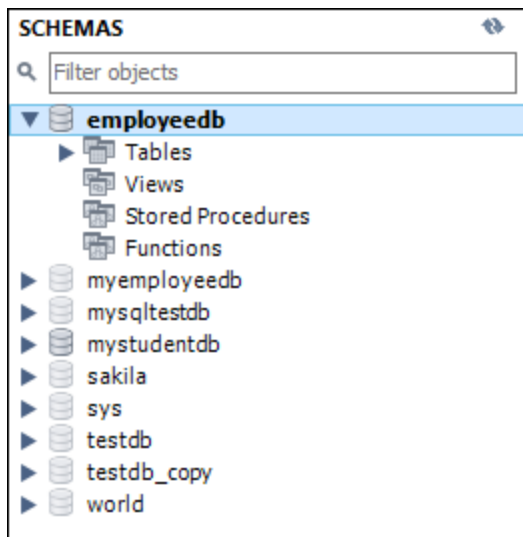
Как да създадете тригер BEFORE UPDATE Trigger в MySQL workbench?

За да създадем тригер BEFORE UPDATE с помощта на MySQL workbench , първо трябва да го стартираме и след това да влезем с потребителското име и парола:

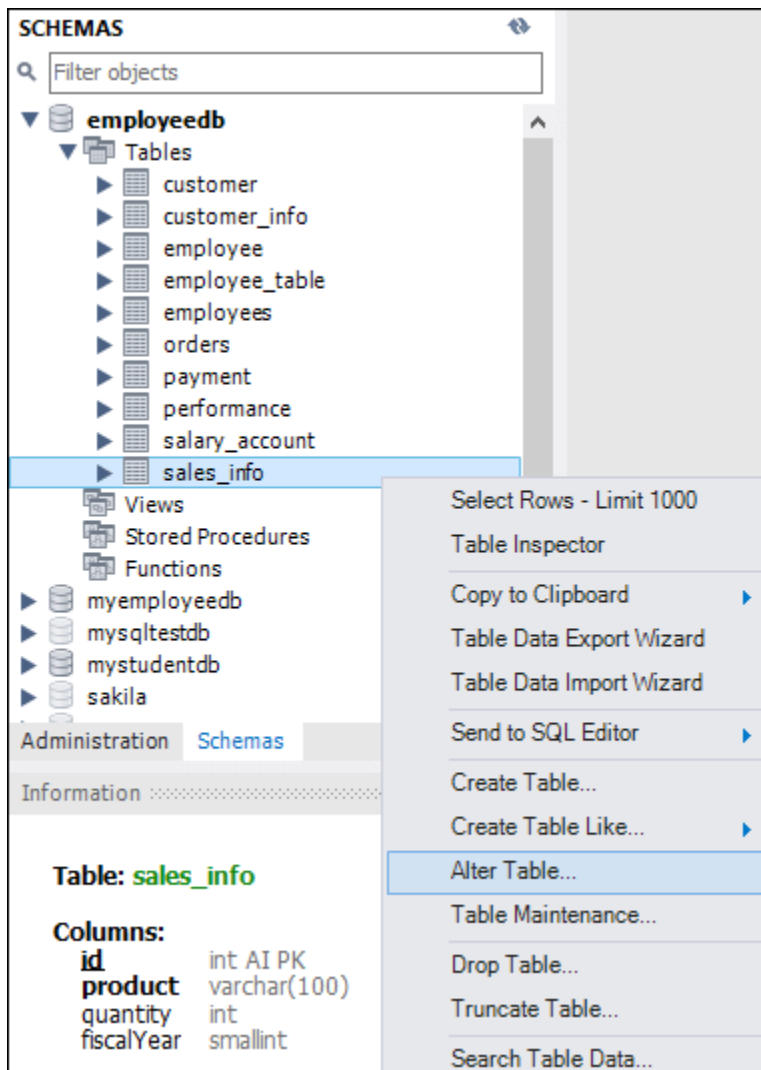


Сега направете следните стъпки за създаване на тригер BEFORE UPDATE:

1. Отидете в раздела **Navigator** и щракнете върху **менюто Shemas** , което съдържа всички налични бази данни в MySQL сървъра.
2. Изберете базата данни (например, **employeedb**), щракнете два пъти върху нея и покажете **подменюто**, съдържащо таблици, изгледи, функции и съхранени процедури.



3. Разгънете **подменюто Tables** и изберете таблицата, за която искате да създадете тригер. След като изберете таблица, щракнете с десния бутон върху избраната таблица (например **sales_info**) и след това щракнете върху опцията **Alter Table** .



4. Щракването върху опцията за **Alter Table** дава екрана:

Table Name: Schema: **employeeedb**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	INT	☒	☒	☐	☐	☐	☐	☒	☐	

Column Name: Data Type:

Charset/Collation: Default:

Comments:

Storage: ☐ Virtual ☐ Stored

☐ Primary Key ☐ Not Null ☐ Unique

☐ Binary ☐ Unsigned ☐ Zero Fill

☐ Auto Increment ☐ Generated

Columns Indexes Foreign Keys **Triggers** Partitioning Options

Apply Revert

5. Сега щракнете върху **раздела Trigger** , показан в червеното правоъгълно поле на предишния раздел, след което изберете Timing/Event **BEFORE UPDATE**. Ще забележим, че има бутон с икона (+) за добавяне на тригер. Щраквайки върху този бутон, ще получим код по подразбиране при задействане въз основа на избора на Време/Събитие:

Table Name: Schema: **employeeedb**

Charset/Collation: Engine:

Comments:

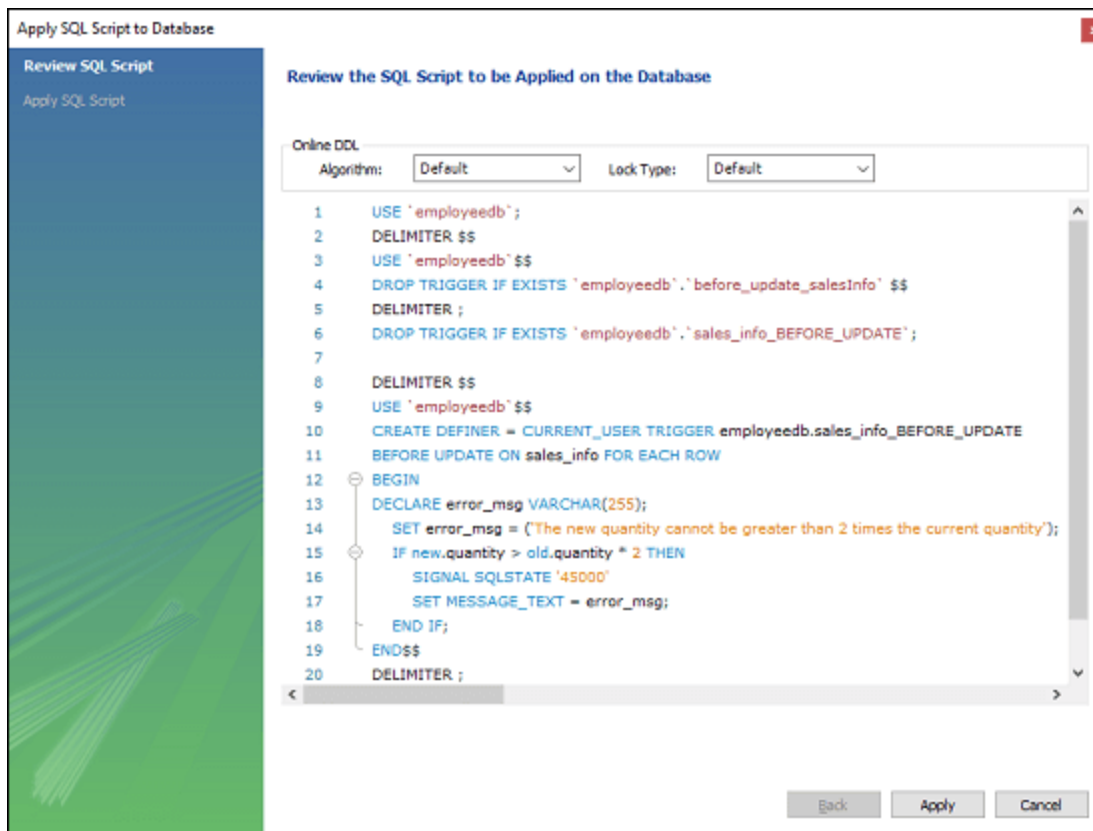
BEFORE INSERT
AFTER INSERT
▼ BEFORE UPDATE ☒ **+**
sales_info_BEFORE_UPDATE
AFTER UPDATE
BEFORE DELETE
AFTER DELETE

```
1 • CREATE DEFINER = CURRENT_USER TRIGGER employeeedb.sales_info_BEFORE_UPDATE
2 BEFORE UPDATE ON sales_info FOR EACH ROW
3 BEGIN
4
5 END
6
```

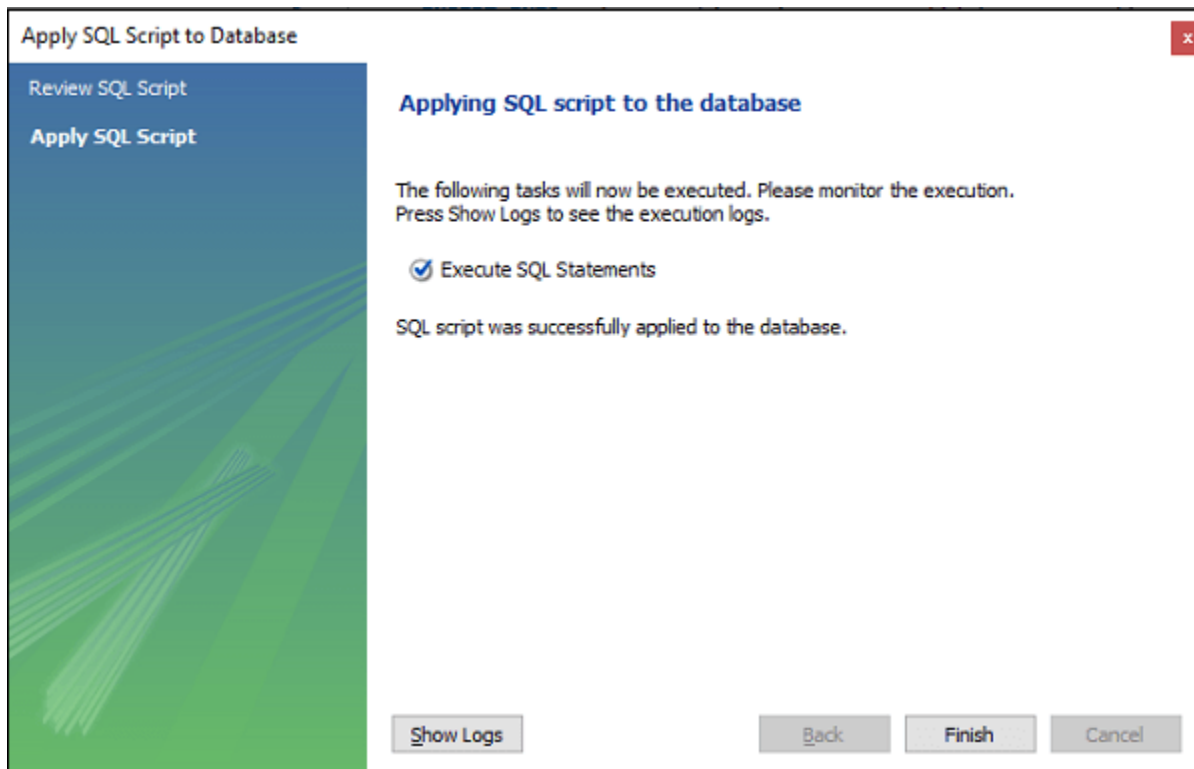
Columns Indexes Foreign Keys **Triggers** Partitioning Options

Apply Revert

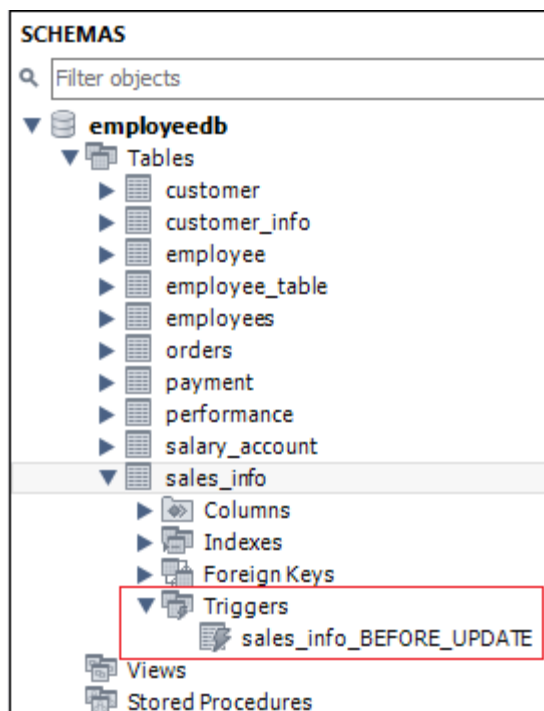
6. Сега попълнете тригерния код, прегледайте ги още веднъж и ако не бъде намерена грешка, щракнете върху бутона **Apply** .



7. След като щракнете върху бутона **Apply**, щракнете върху бутона **Finish** за завършване.



8. Ако влезем менюто на схемата, ще видим тригера **sales_info_before_update** под таблицата **sales_info**:



MySQL AFTER UPDATE TRIGGER

Тригерът **AFTER UPDATE** в MySQL се извиква автоматично всеки път, когато се задейства събитие UPDATE в таблицата, свързана с тригерите. Как да създадем тригер AFTER UPDATE със съответния синтаксис.

Синтаксис

Създаване на тригер **AFTER UPDATE** в MySQL:

```
CREATE TRIGGER trigger_name  
AFTER UPDATE  
ON table_name FOR EACH ROW  
trigger_body ;
```

Параметри на синтаксиса на тригера **AFTER UPDATE**:

- Първо, ще посочим **trigger name / името на тригера** , който искаме да създадем. Тя трябва да бъде уникално в рамките на схемата.
- Второ, ще посочим **trigger action time / времето за действие на тригера** , което трябва да бъде **AFTER UPDATE**. Този тригер ще бъде извикан след всеки ред от промени в таблицата.
- Трето, ще посочим **table name / името на таблицата** , към която е свързан тригерът. Трябва да бъде написано след **ON**. Ако не сме указали името на таблицата, тригерът няма да съществува.
- Накрая ще посочим **table name / тялото на тригера** , което съдържа оператор за изпълнение, когато тригерът е активиран.

Ако искаме да изпълним повече от един оператор, ще използваме блока **BEGIN END** , който съдържа набор от SQL заявки, за да дефинираме логиката за тригера.

Синтаксис:

```
DELIMITER $$  
CREATE TRIGGER trigger_name AFTER UPDATE  
ON table_name FOR EACH ROW  
BEGIN  
    variable declarations  
    trigger code  
END$$  
DELIMITER ;
```

Ограничения

- Имаме достъп до OLD редове, но не можем да ги актуализираме.
- Имаме достъп до NEW редове, но не можем да ги актуализираме.
- Не можем да създадем тригер **AFTER UPDATE** на **VIEW**.

Пример за тригер **AFTER UPDATE**

Как да създадем тригер **AFTER UPDATE** с помощта на оператора CREATE TRIGGER в MySQL?

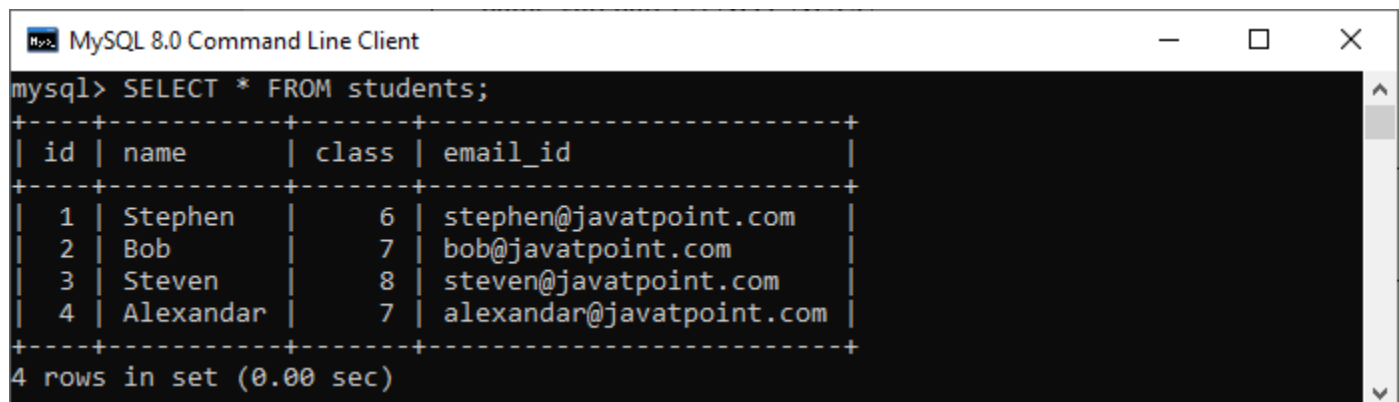
Създаваме таблица с име **students**, за да съхраняваме информацията за студента, както следва:

```
CREATE TABLE students(  
    id int NOT NULL AUTO_INCREMENT,  
    name varchar(45) NOT NULL,  
    class int NOT NULL,  
    email_id varchar(65) NOT NULL,  
    PRIMARY KEY (id)  
);
```

След това ще вмъкнем някои записи в тази таблица:

```
INSERT INTO students (name, class, email_id)  
VALUES ('Stephen', 6, 'stephen@javatpoint.com'),  
('Bob', 7, 'bob@javatpoint.com'),  
('Steven', 8, 'steven@javatpoint.com'),  
('Alexandar', 7, 'alexandar@javatpoint.com');
```

Изпълнете заявката **SELECT**, за да видите данните от таблицата.



```
MySQL 8.0 Command Line Client  
mysql> SELECT * FROM students;  
+----+-----+-----+-----+  
| id | name   | class | email_id                |  
+----+-----+-----+-----+  
| 1  | Stephen | 6     | stephen@javatpoint.com  |  
| 2  | Bob     | 7     | bob@javatpoint.com      |  
| 3  | Steven  | 8     | steven@javatpoint.com   |  
| 4  | Alexandar | 7     | alexandar@javatpoint.com |  
+----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

Трето, ще създадем друга таблица с име **students_log** , която съхранява актуализираната информация в избрания потребител.

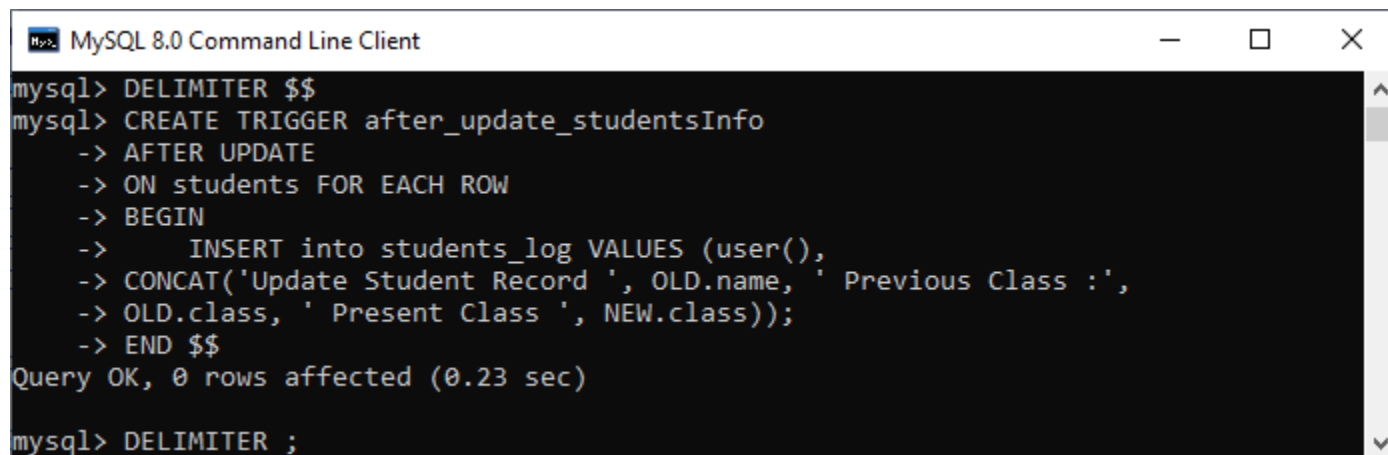
```
CREATE TABLE students_log(  
  user varchar(45) NOT NULL,  
  descriptions varchar(65) NOT NULL  
);
```

След това ще създадем **тригер AFTER UPDATE**, който повишава всички ученици в следващия клас , т.е. 6 ще бъде 7, 7 ще бъде 8 и т.н. Всеки път, когато се извършва актуализация на един ред в таблицата „students“, нов ред ще бъде вмъкнат в таблицата „students_log “. Тази таблица съхранява **текущия потребителски идентификатор** и **описание** относно текущата актуализация.

```
DELIMITER $$
```

```
CREATE TRIGGER after_update_studentsInfo  
AFTER UPDATE  
ON students FOR EACH ROW  
BEGIN  
  INSERT into students_log VALUES (user(),  
  CONCAT('Update Student Record ', OLD.name, ' Previous Class :',  
  OLD.class, ' Present Class ', NEW.class));  
END $$
```

```
DELIMITER ;
```



```
MySQL 8.0 Command Line Client  
mysql> DELIMITER $$  
mysql> CREATE TRIGGER after_update_studentsInfo  
-> AFTER UPDATE  
-> ON students FOR EACH ROW  
-> BEGIN  
->   INSERT into students_log VALUES (user(),  
->   CONCAT('Update Student Record ', OLD.name, ' Previous Class :',  
->   OLD.class, ' Present Class ', NEW.class));  
-> END $$  
Query OK, 0 rows affected (0.23 sec)  
mysql> DELIMITER ;
```

В този тригер първо сме посочили името на тригера **after_update_studentsInfo** . След това посочваме задействащото събитие. Трето, уточнихме името на таблицата, към която е свързан тригерът. И накрая, написахме логиката на тригера вътре в тялото на тригера, която извършва актуализиране в таблицата „students“ и съхранява информацията в таблицата „students_log“.

Как да извикам тригера AFTER UPDATE?

Първо, ще актуализираме таблицата „students“ с помощта на следните изрази, които извикват създадения по-горе тригер:

```
UPDATE students SET class = class + 1;
```

След това ще поискаме данни от **таблиците students** и **students_log** . Виждаме, че таблицата е актуализирана след изпълнението на заявката.

```
MySQL 8.0 Command Line Client
mysql> UPDATE students SET class = class + 1;
Query OK, 4 rows affected (0.15 sec)
Rows matched: 4  Changed: 4  Warnings: 0

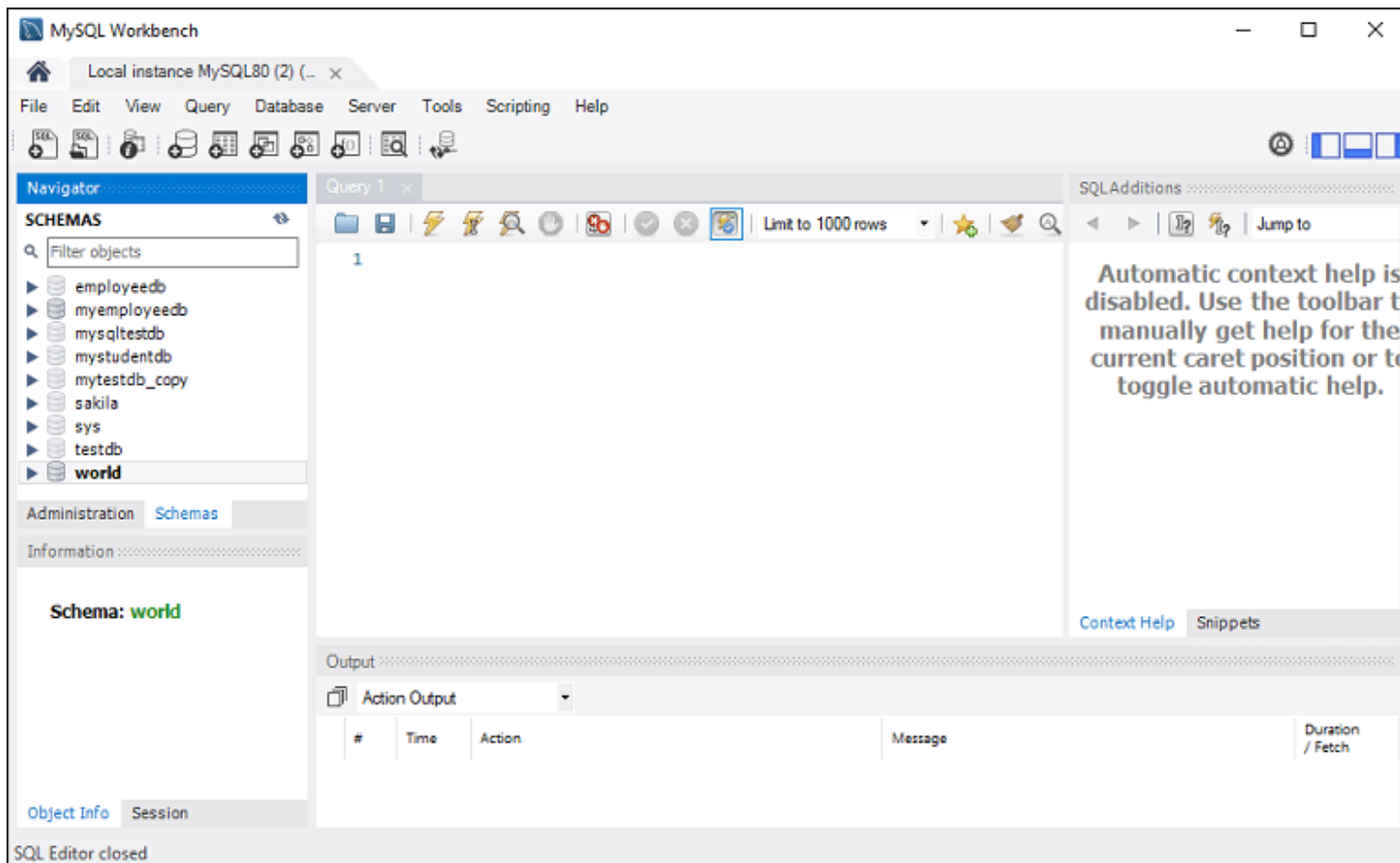
mysql> SELECT * FROM students;
+----+-----+-----+-----+
| id | name   | class | email_id |
+----+-----+-----+-----+
| 1  | Stephen | 7     | stephen@javatpoint.com |
| 2  | Bob    | 8     | bob@javatpoint.com     |
| 3  | Steven | 9     | steven@javatpoint.com  |
| 4  | Alexandar | 8     | alexandar@javatpoint.com |
+----+-----+-----+-----+
4 rows in set (0.09 sec)
```

Отново ще поискаме данни от таблицата **students_log**, която пази текущия потребителски идентификатор и описание относно текущата актуализация.

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM students_log;
+-----+-----+
| user      | descriptions |
+-----+-----+
| root@localhost | Update Student Record Stephen Previous Class :6 Present Class 7 |
| root@localhost | Update Student Record Bob Previous Class :7 Present Class 8 |
| root@localhost | Update Student Record Steven Previous Class :8 Present Class 9 |
| root@localhost | Update Student Record Alexandar Previous Class :7 Present Class 8 |
+-----+-----+
4 rows in set (0.00 sec)
```

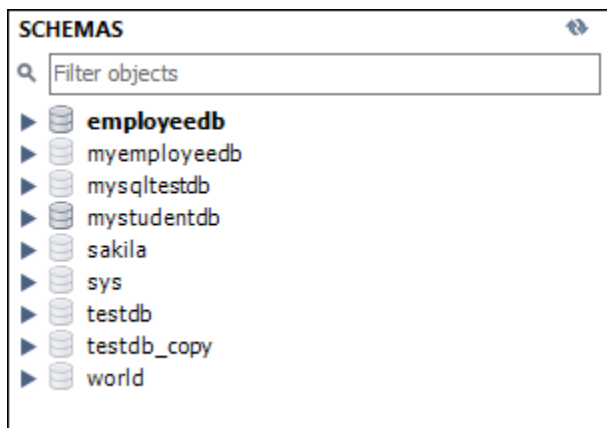
Как да създадете тригер AFTER UPDATE в MySQL workbench?

За да създадем тригер AFTER UPDATE в работната среда, първо стартираме **MySQL Workbench** и влизаме с потребителското име и парола:

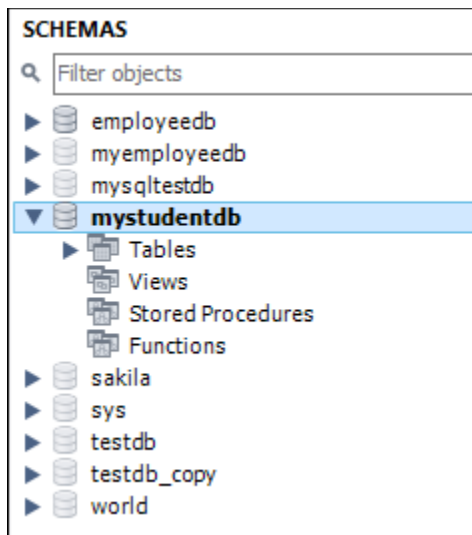


Сега направете следните стъпки, за да създадете тригер AFTER UPDATE:

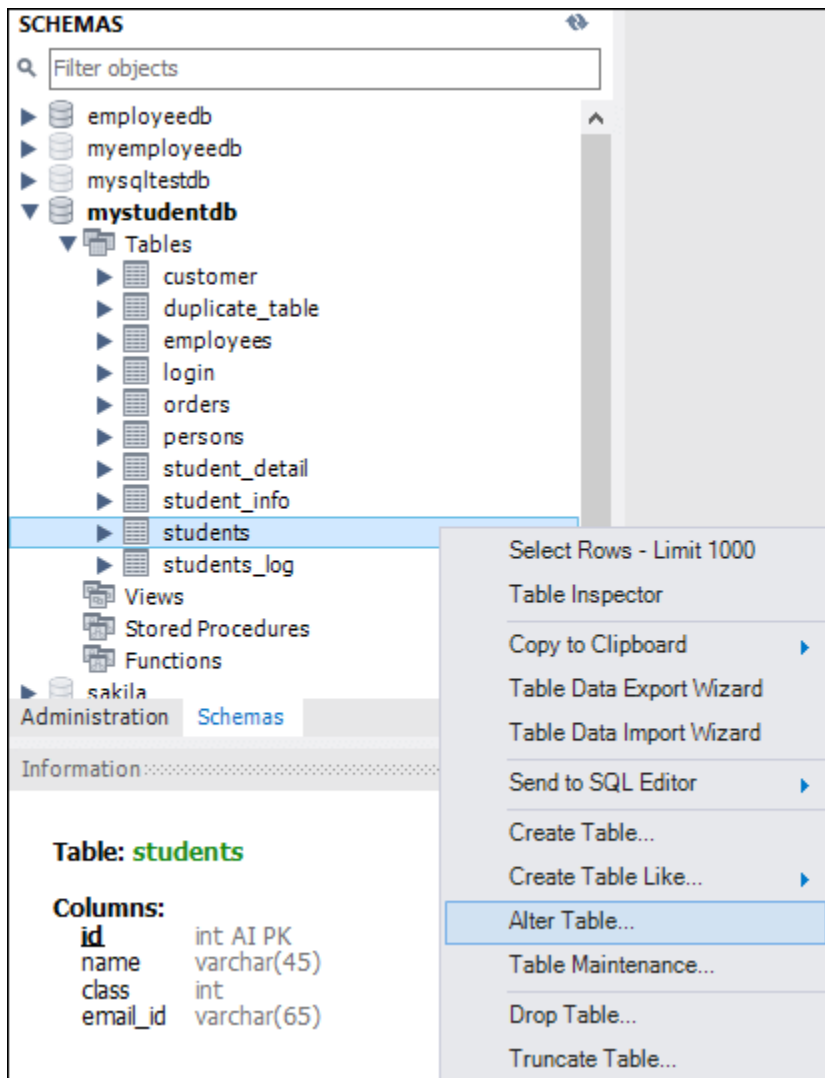
1. Отидете в раздела **Navigator** и щракнете върху менюто **Schemas** . Той ще покаже всички налични бази данни в MySQL сървъра за бази данни.



2. Изберете базата данни (например **mystudentdb**). След това кликнете два пъти върху избраната схема. Той показва **подменюто**, съдържащо таблици, изгледи, функции и съхранени процедури.



3. Разгънете подменюто **Tables** и изберете таблица, върху която искате да създадете тригер. След това щракнете с десния бутон върху избраната таблица (**например** students) и щракнете върху опцията **Alter Table**.



4. Щракването върху опцията за **Alter Table** дава екрана:

Table Name: Schema: **mystudentdb**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	INT	☒	☒	☐	☐	☐	☐	☒	☐	

Column Name: Data Type:

Charset/Collation: Default:

Comments:

Storage: ☐ Virtual ☐ Stored

☐ Primary Key ☐ Not Null ☐ Unique

☐ Binary ☐ Unsigned ☐ Zero Fill

☐ Auto Increment ☐ Generated

Columns Indexes Foreign Keys **Triggers** Partitioning Options

Apply Revert

5. Сега щракнете върху раздела **Trigger** , **показан в червеното правоъгълно поле** на предишния раздел , след което изберете **Време/Събитие** AFTER UPDATE. Ще забележим, че има **бутон с икона (+)** за добавяне на тригер. Щраквайки върху този бутон, ще получим код по подразбиране на тригера въз основа на избора на Време/Събитие:

Table Name: Schema: **mystudentdb**

Charset/Collation: Engine:

Comments:

BEFORE INSERT
AFTER INSERT
BEFORE UPDATE
▼ AFTER UPDATE
 student_update_trigger
BEFORE DELETE
AFTER DELETE

```

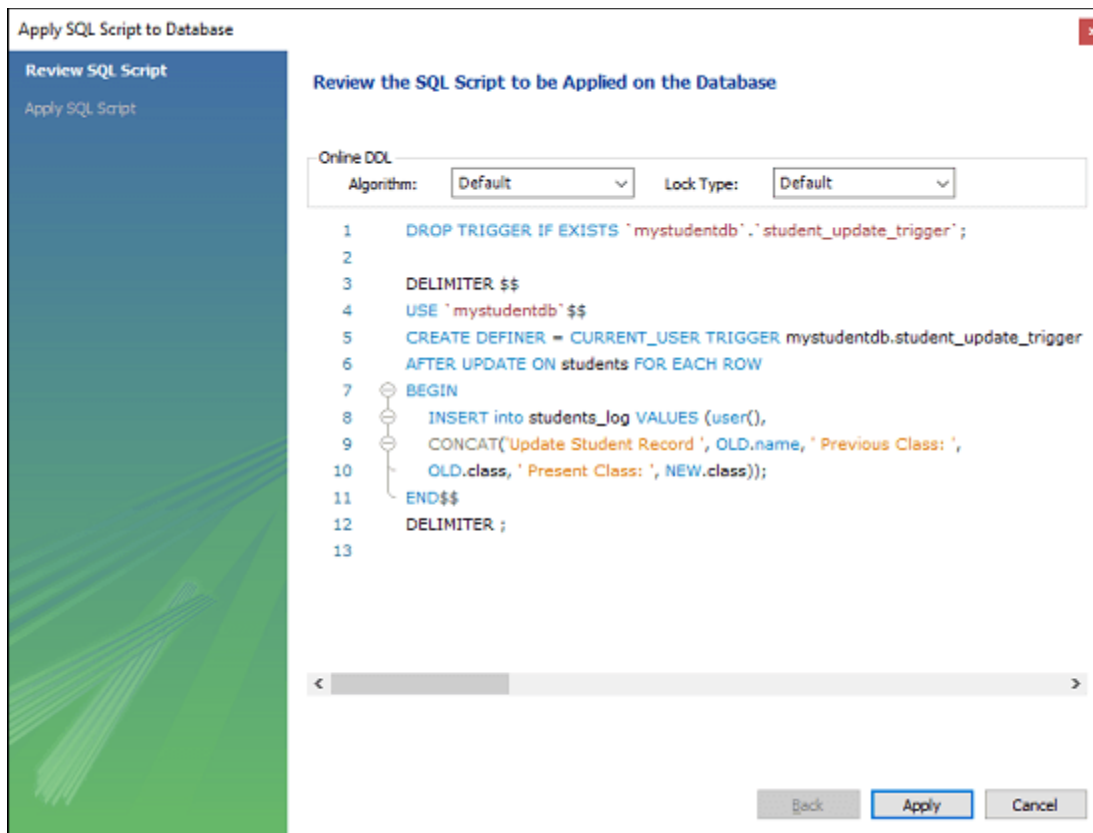
1 • CREATE DEFINER = CURRENT_USER TRIGGER mystudentdb.student_update_trigger
2   AFTER UPDATE ON students FOR EACH ROW
3   BEGIN
4
5   END
6

```

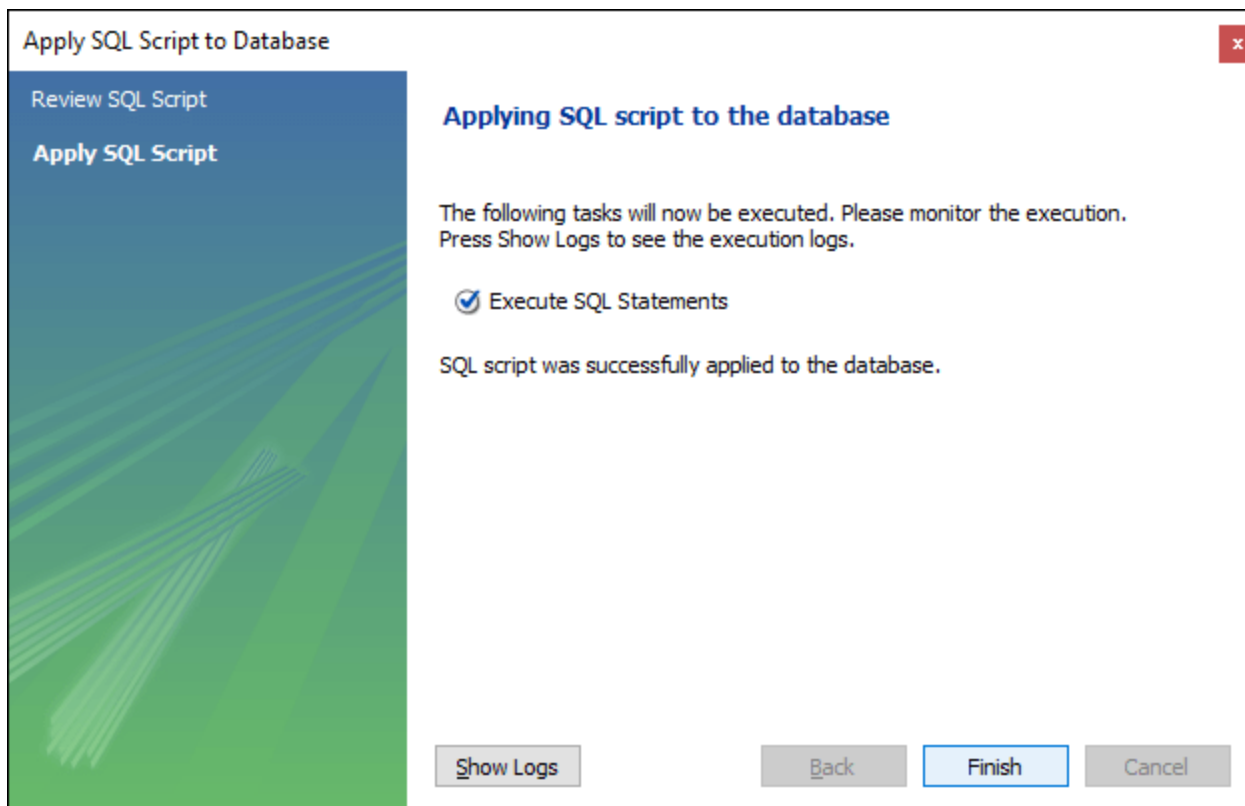
Columns Indexes Foreign Keys **Triggers** Partitioning Options

Apply Revert

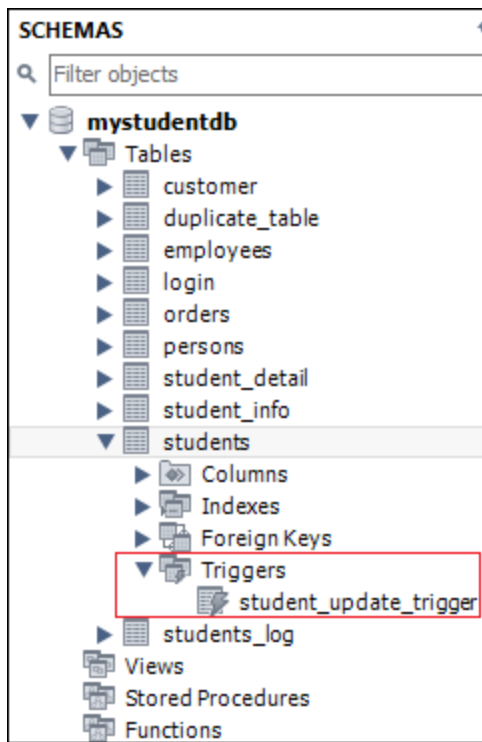
6. Сега попълнете тригерния код, прегледайте ги още веднъж и ако не бъде намерена грешка, щракнете върху бутона **Apply** .



7. След като щракнете върху бутона Apply, щракнете върху **бутона Finish**, за да завършите процеса.



8. Ако погледнем менюто на схемата, можем да видим **student_update_trigger** под таблицата " **students**", както следва:



MySQL BEFORE DELETE Trigger

BEFORE DELETE Trigger в MySQL се извиква автоматично всеки път, когато се задейства операция за изтриване на таблицата. Как да създадем тригер BEFORE DELETE със съответния синтаксис.

Синтаксис

Синтаксис за създаване на тригер BEFORE DELETE в MySQL:

```
CREATE TRIGGER trigger_name  
BEFORE DELETE  
ON table_name FOR EACH ROW  
Trigger_body ;
```

Параметри на тригера BEFORE DELETE:

- Първо, ще посочим името на тригера, който искаме да създадем. Тя трябва да бъде уникално в рамките на схемата.
- Второ, ще посочим времето за действие на тригера, което трябва да бъде **BEFORE DELETE**. Този тригер ще бъде извикан преди всеки ред от промени да се случи в таблицата.
- Трето, ще посочим името на таблица, към която е свързан тригерът. Трябва да се напише след ключовата дума **ON**. Ако не посочим името на таблицата, тригерът няма да съществува.
- Накрая ще посочим оператора за изпълнение, когато тригерът е активиран.

Ако искаме да изпълним множество изрази, ще използваме блока **BEGIN END**, който съдържа набор от заявки за дефиниране на логиката за тригера. Вижте синтаксиса по-долу:

```
DELIMITER $$  
CREATE TRIGGER trigger_name BEFORE DELETE  
ON table_name FOR EACH ROW  
BEGIN  
    variable declarations  
    trigger code  
END$$  
DELIMITER ;
```

Ограничения

- Имаме достъп до **OLD** редове, но не можем да ги актуализираме в тригер **BEFORE DELETE**.
- Нямаме достъп до **NEW** редове. Това е така, защото не съществува нов ред.
- Не можем да създадем тригер **BEFORE DELETE** на **VIEW**.

Пример за задействане **BEFORE DELETE**

Как да създадем тригер **BEFORE DELETE** с помощта на оператора **CREATE TRIGGER** в MySQL?

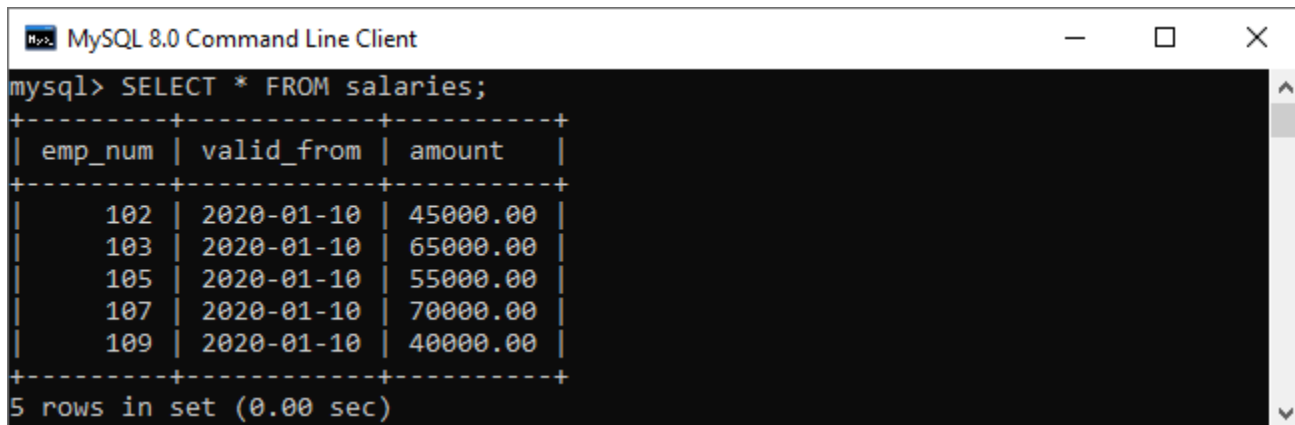
Създаваме таблица с име **salaries**, за да съхраняваме информацията за заплатата на служител, както следва:

```
CREATE TABLE salaries (  
    emp_num INT PRIMARY KEY,  
    valid_from DATE NOT NULL,  
    amount DEC(8 , 2 ) NOT NULL DEFAULT 0  
);
```

След това ще вмъкнем някои записи в тази таблица, като използваме израза по-долу:

```
INSERT INTO salaries (emp_num, valid_from, amount)  
VALUES  
    (102, '2020-01-10', 45000),  
    (103, '2020-01-10', 65000),  
    (105, '2020-01-10', 55000),  
    (107, '2020-01-10', 70000),  
    (109, '2020-01-10', 40000);
```

Изпълнете заявката **SELECT**, за да видите данните от таблицата.



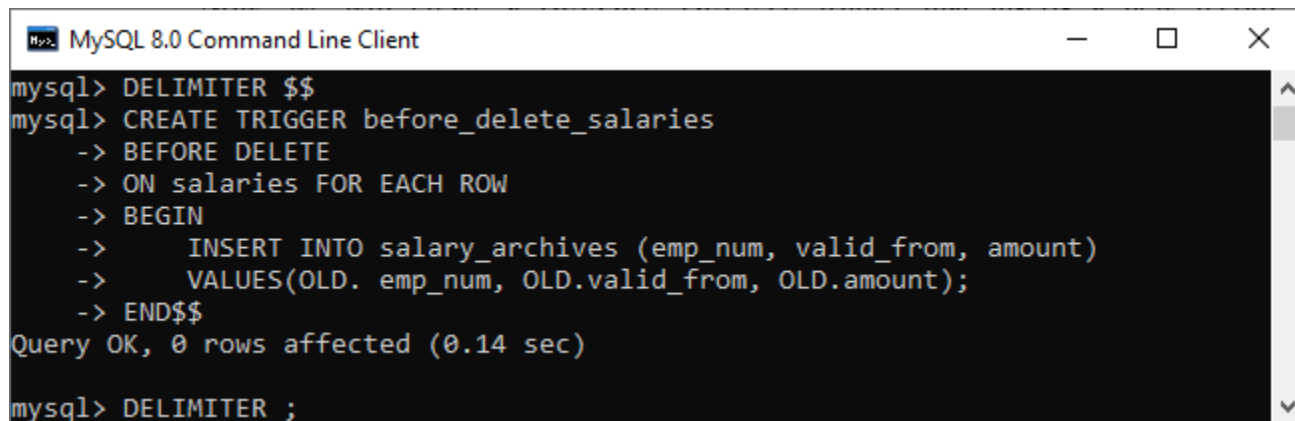
```
MySQL 8.0 Command Line Client  
mysql> SELECT * FROM salaries;  
+-----+-----+-----+  
| emp_num | valid_from | amount |  
+-----+-----+-----+  
|      102 | 2020-01-10 | 45000.00 |  
|      103 | 2020-01-10 | 65000.00 |  
|      105 | 2020-01-10 | 55000.00 |  
|      107 | 2020-01-10 | 70000.00 |  
|      109 | 2020-01-10 | 40000.00 |  
+-----+-----+-----+  
5 rows in set (0.00 sec)
```

Трето, ще създадем друга таблица с име **salary_archives**, която съхранява информацията за изтритата заплата.

```
CREATE TABLE salary_archives (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  emp_num INT,  
  valid_from DATE NOT NULL,  
  amount DEC(18, 2) NOT NULL DEFAULT 0,  
  deleted_time TIMESTAMP DEFAULT NOW()  
);
```

След това ще създадем тригер **BEFORE DELETE**, който вмъква нов запис в таблицата **salary_archives**, преди редът да бъде изтрит от таблицата **salaries**.

```
DELIMITER $$  
  
CREATE TRIGGER before_delete_salaries  
BEFORE DELETE  
ON salaries FOR EACH ROW  
BEGIN  
  INSERT INTO salary_archives (emp_num, valid_from, amount)  
  VALUES(OLD. emp_num, OLD.valid_from, OLD.amount);  
END$$  
  
DELIMITER ;
```



```
MySQL 8.0 Command Line Client  
mysql> DELIMITER $$  
mysql> CREATE TRIGGER before_delete_salaries  
-> BEFORE DELETE  
-> ON salaries FOR EACH ROW  
-> BEGIN  
->   INSERT INTO salary_archives (emp_num, valid_from, amount)  
->   VALUES(OLD. emp_num, OLD.valid_from, OLD.amount);  
-> END$$  
Query OK, 0 rows affected (0.14 sec)  
  
mysql> DELIMITER ;
```

В този тригер първо сме посочили името на тригера **before_delete_salaries**. След това посочихме задействащото събитие. Трето, уточнихме името на таблицата, към която е свързан тригерът. И накрая, написахме логиката на тригера в тялото на тригера, която вмъква изтрения ред в таблицата **salary_archives**.

Как да извикам тригера BEFORE DELETE?

Нека тестваме създадения по-горе тригер BEFORE DELETE и как можем да ги извикаме. Така че първо ще премахнем ред от таблицата със заплатите:

```
mysql> DELETE FROM salaries WHERE emp_num = 105;
```

Второ, ще направим заявка за данни от таблицата **salary_archives**, за да проверим дали създаденият по-горе тригер е извикан или не, чрез използване на оператора **select**:

```
SELECT * FROM salary_archives;
```

След като изпълним оператор, можем да видим, че тригерът е извикан успешно и е вмъкнал нов запис в таблицата **salary_archives**.

```
MySQL 8.0 Command Line Client
mysql> DELETE FROM salaries WHERE emp_num = 105;
Query OK, 1 row affected (0.13 sec)

mysql> SELECT * FROM salary_archives;
+-----+-----+-----+-----+-----+
| id | emp_num | valid_from | amount | deleted_time |
+-----+-----+-----+-----+-----+
| 1 | 105 | 2020-01-10 | 55000.00 | 2020-11-20 13:16:29 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Трето, ще премахнем всички редове от таблицата със заплатите:

```
DELETE FROM salaries;
```

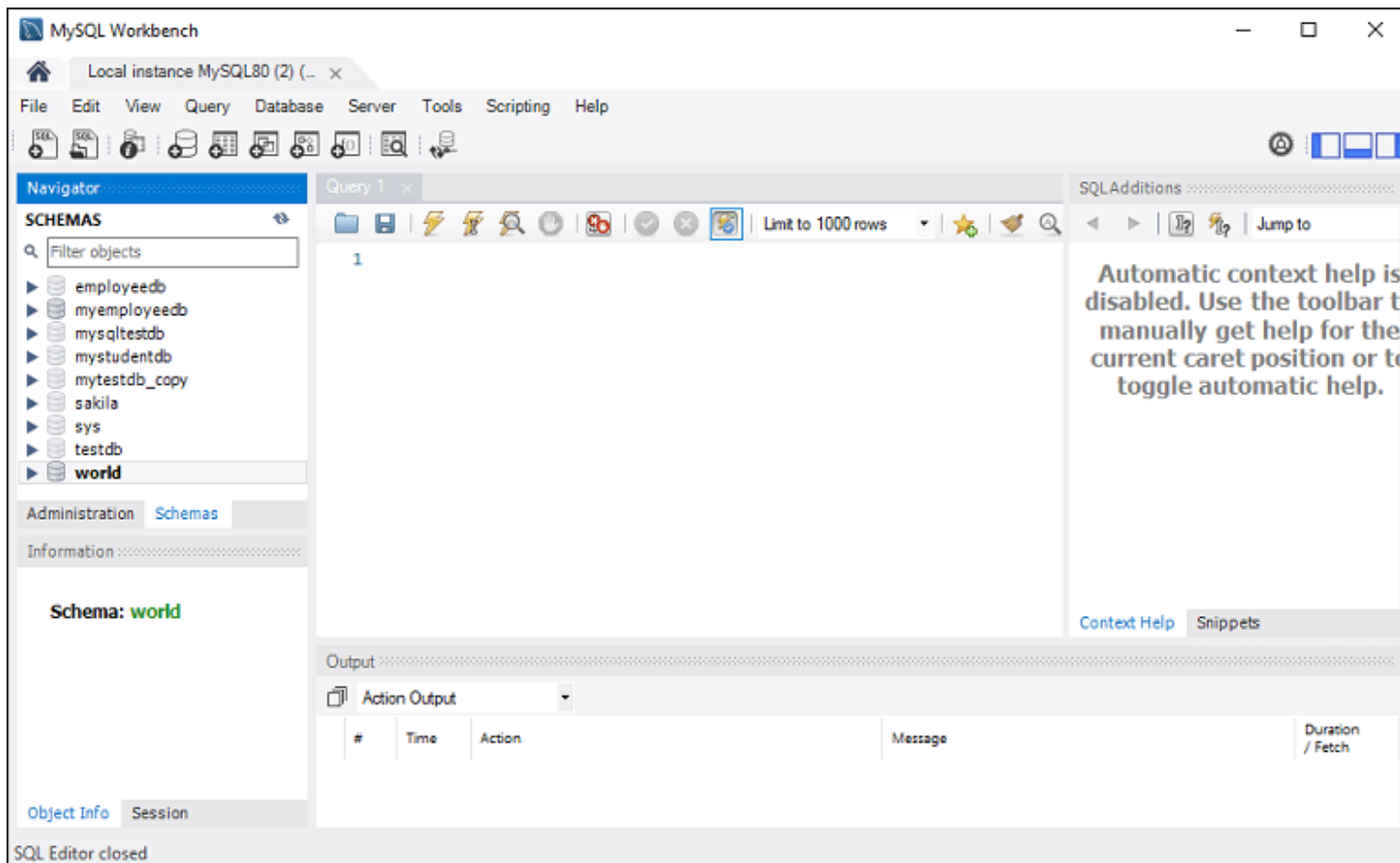
И накрая, отново ще поискаме данни от таблицата **salary_archives**. Тригерът беше извикан четири пъти, защото командата **DELETE** премахна четири записа от таблицата със заплатите. Вижте резултата по-долу:

```
MySQL 8.0 Command Line Client
mysql> DELETE FROM salaries;
Query OK, 4 rows affected (0.18 sec)

mysql> SELECT * FROM salary_archives;
+-----+-----+-----+-----+-----+
| id | emp_num | valid_from | amount | deleted_time |
+-----+-----+-----+-----+-----+
| 1 | 105 | 2020-01-10 | 55000.00 | 2020-11-20 13:16:29 |
| 2 | 102 | 2020-01-10 | 45000.00 | 2020-11-20 13:17:59 |
| 3 | 103 | 2020-01-10 | 65000.00 | 2020-11-20 13:17:59 |
| 4 | 107 | 2020-01-10 | 70000.00 | 2020-11-20 13:17:59 |
| 5 | 109 | 2020-01-10 | 40000.00 | 2020-11-20 13:17:59 |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

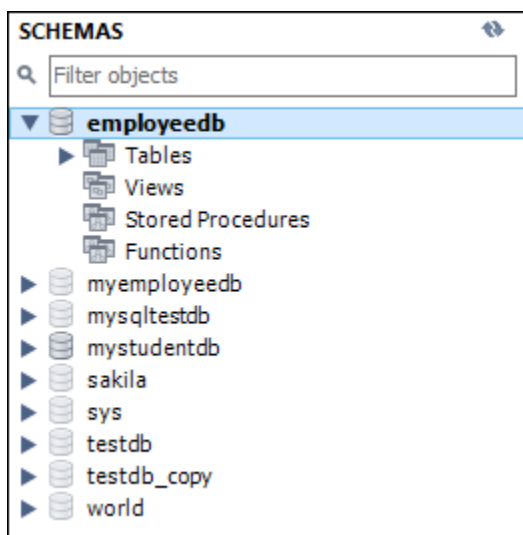
Как да създадете BEFORE DELETE Trigger в MySQL workbench?

За да създадем тригер **BEFORE DELETE** с помощта на MySQL workbench , първо трябва да го стартираме и след това да влезем с потребителското име и парола:

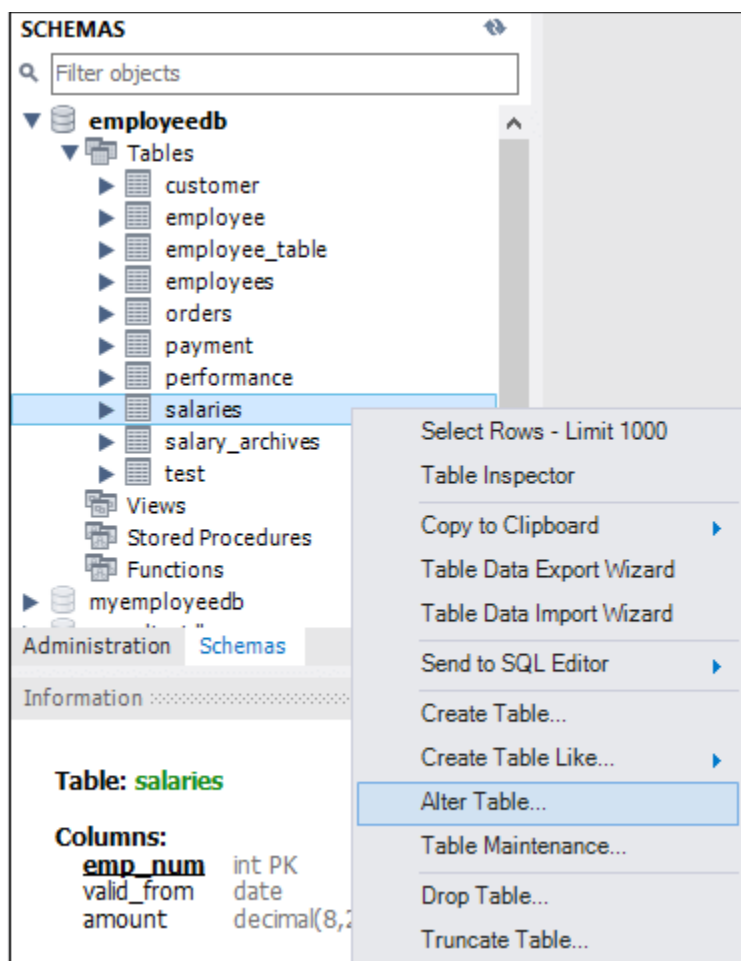


Сега направете следните стъпки за създаване на тригер **BEFORE DELETE**:

1. Отидете в раздела **Navigator** и щракнете върху **менюто Shemas** , което съдържа всички налични бази данни в MySQL сървъра.
2. Изберете базата данни (например **employeedb**), щракнете двукратно върху нея. Ще покаже **подменюто** , което съдържа таблици, изгледи, функции и съхранени процедури.



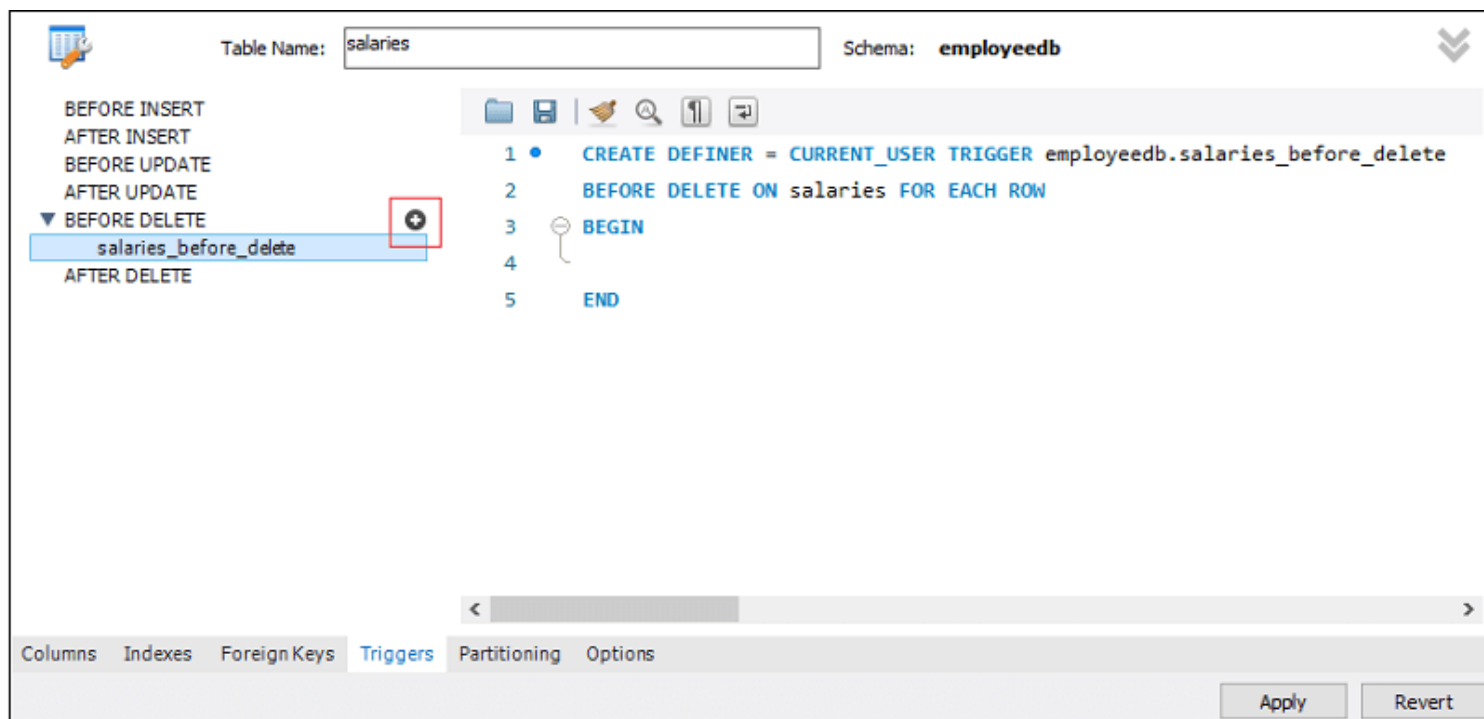
3. Разгънете **подменюто Tables** и изберете таблицата, за която искате да създадете тригер . След като изберете таблица, щракнете с десния бутон върху избраната таблица (например **salaries**) и след това щракнете върху опцията **Alter Table**:



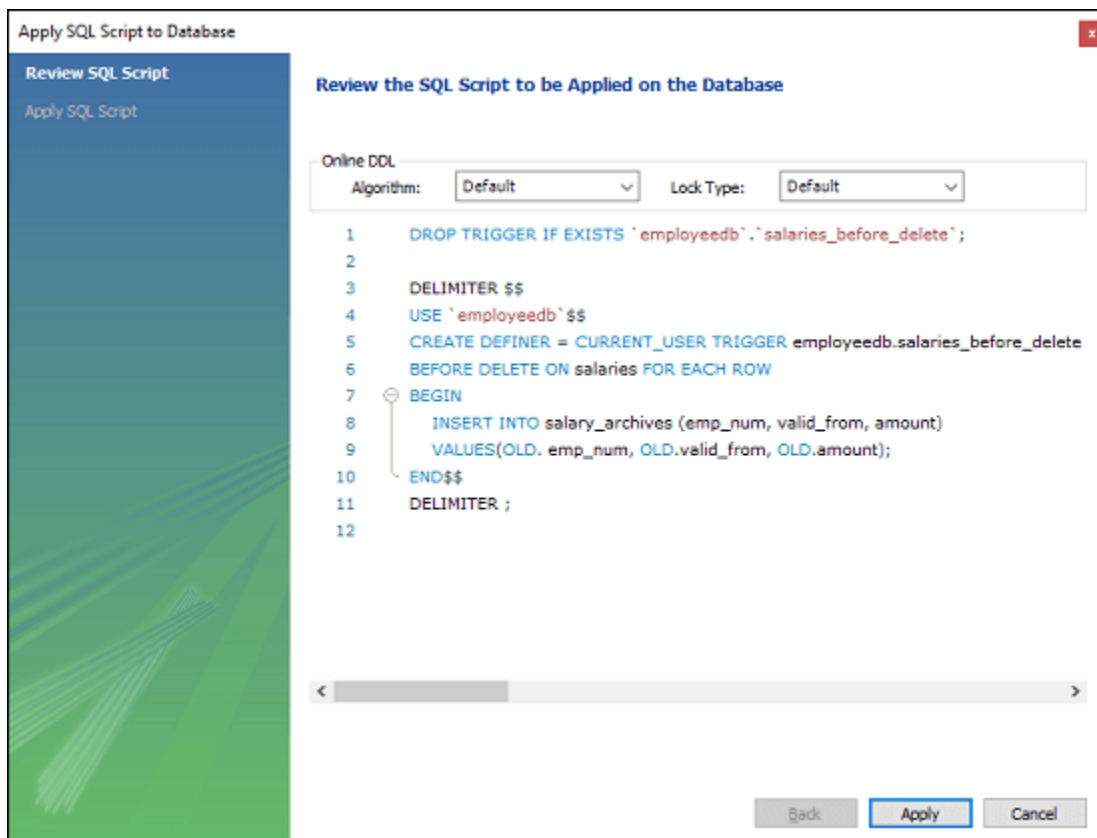
4. Щракването върху опцията за **Alter Table** дава екрана:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
emp_num	INT	☒	☒	☐	☐	☐	☐	☐	☐	

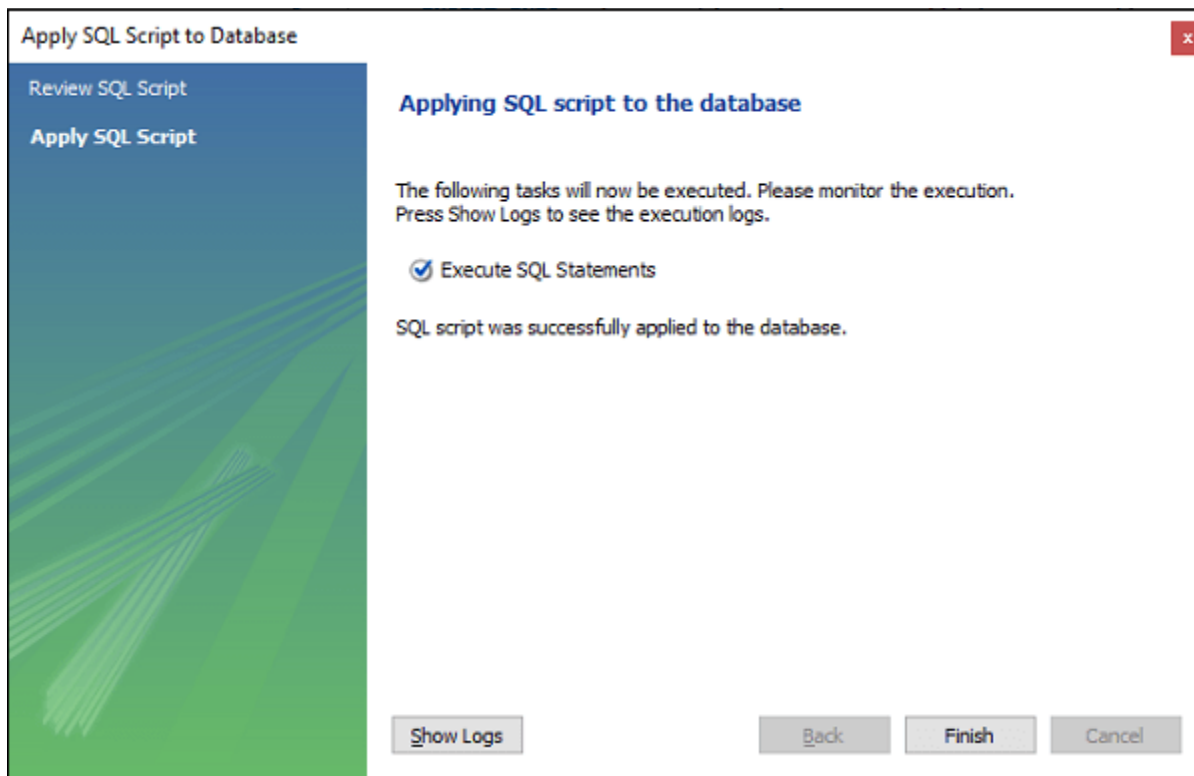
5. Сега щракнете върху **раздела Trigger**, показан в червеното правоъгълно поле на предишния раздел, след което изберете **Timing/Event BEFORE DELETE**. Ще забележим, че има бутон с икона (+) за добавяне на тригер. Щраквайки върху този бутон, ще получим код по подразбиране при задействане въз основа на избора на Време/Събитие:



6. Сега попълнете тригерния код, прегледайте ги още веднъж и ако не бъде намерена грешка, щракнете върху бутона **Apply**.



7. След като щракнете върху бутона **Apply**, щракнете върху бутона **Finish** за завършване.



8. Ако влезем менюто на схемата, ще видим тригера **salaries_before_trigger** под таблицата със заплатите, както следва:

